

# 지능형 에이전트

# AI 에이전트란?

- AI 에이전트의 정의 및 특징
- 전통적 프로그램과 AI 에이전트의 차이점
- AI 에이전트 동향 및 사례

인공지능



규칙 기반



머신러닝



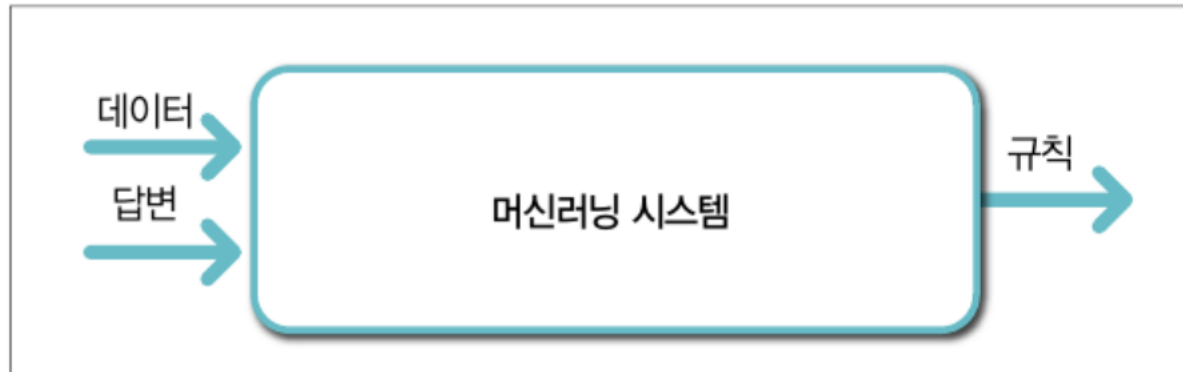
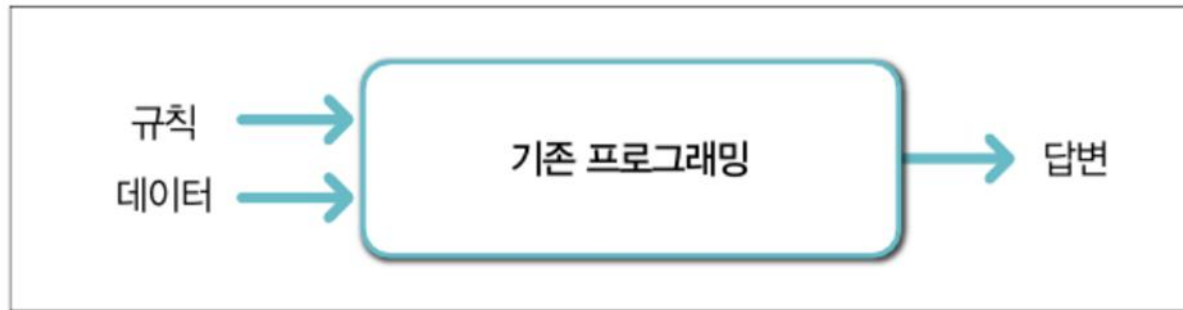
딥러닝



GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍

한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# 머신러닝

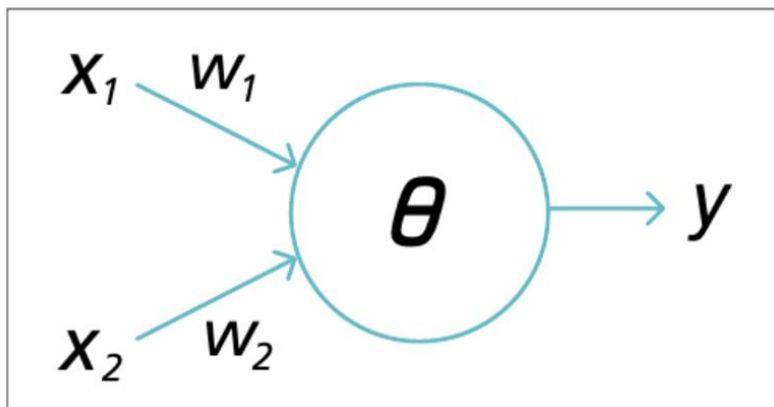


**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**

한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

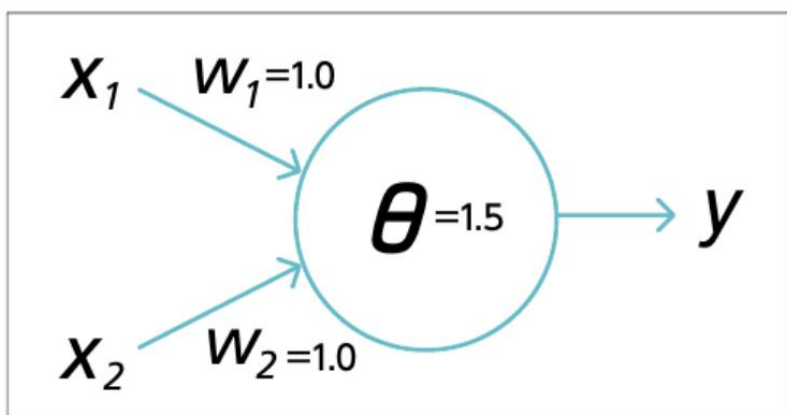


# 퍼셉트론



| 매개변수       | 설명       |
|------------|----------|
| $x_1, x_2$ | 입력(데이터)  |
| $y$        | 출력(답변)   |
| $w_1, w_2$ | 가중치 매개변수 |
| $\theta$   | 임계값      |

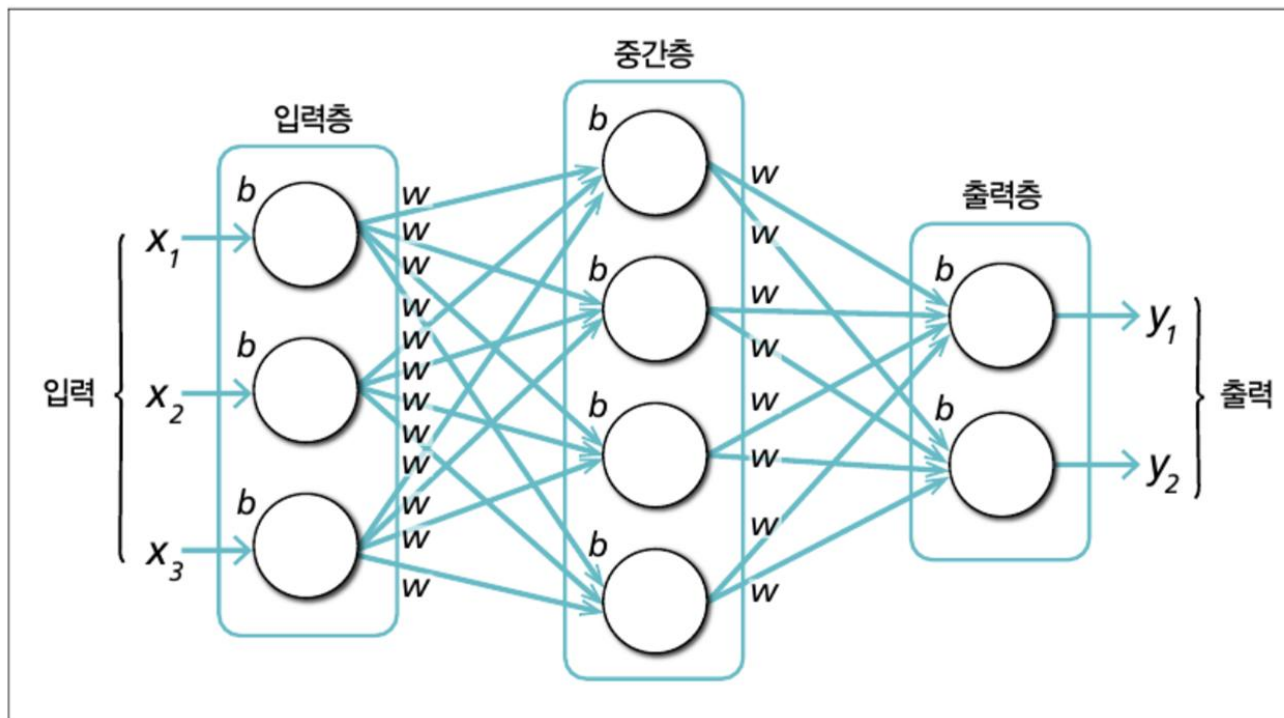
| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 1     | 0     | 0   |
| 0     | 1     | 0   |
| 1     | 1     | 1   |



$x_1 \times w_1 + x_2 \times w_2$ 를 실제로 계산해 보면 다음과 같습니다.

- 입력이  $x_1=0.0, x_2=0.0$ 일 때,  $0.0 \times 1.0 + 0.0 \times 1.0 = 0.0$ (임계값 1.5 이하)이므로 출력은 0이다.
- 입력이  $x_1=1.0, x_2=0.0$ 일 때,  $1.0 \times 1.0 + 0.0 \times 1.0 = 1.0$ (임계값 1.5 이하)이므로 출력은 0이다.
- 입력이  $x_1=0.0, x_2=1.0$ 일 때,  $0.0 \times 1.0 + 1.0 \times 1.0 = 1.0$ (임계값 1.5 이하)이므로 출력은 0이다.
- 입력이  $x_1=1.0, x_2=1.0$ 일 때,  $1.0 \times 1.0 + 1.0 \times 1.0 = 2.0$ (임계값 1.5 이상)이므로 출력은 1이다.

# 딥러닝(신경망)



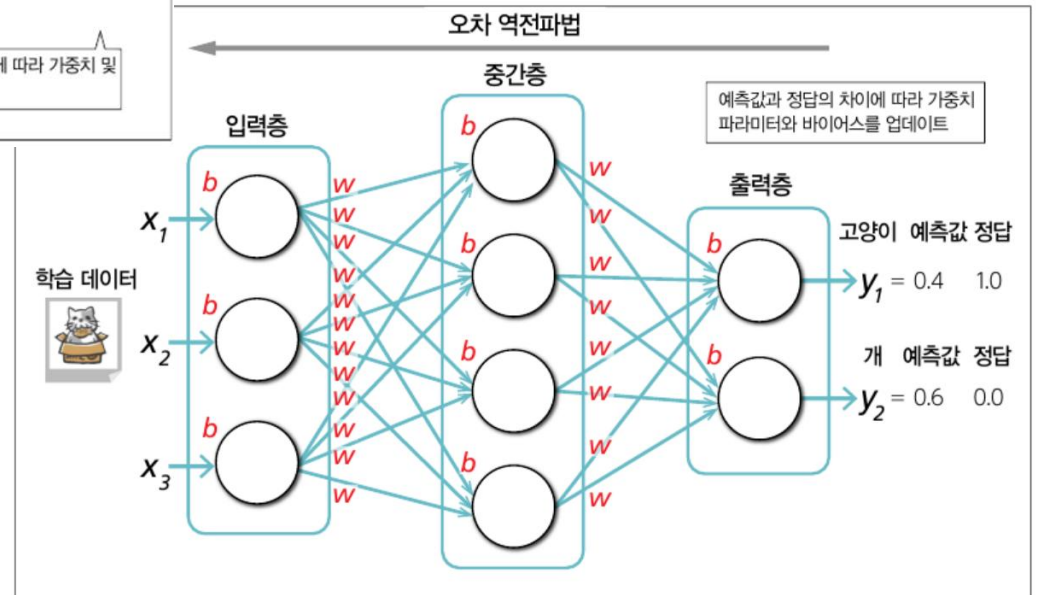
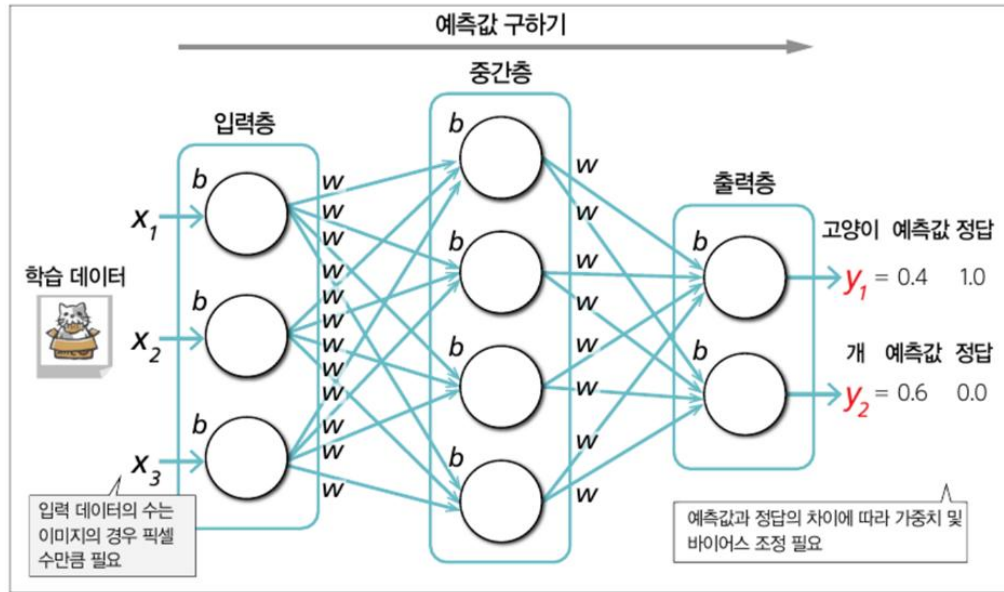
| 매개변수            | 설명       |
|-----------------|----------|
| $x_1, x_2, x_3$ | 입력(데이터)  |
| $y_1, y_2$      | 출력(답변)   |
| $w$             | 가중치 매개변수 |
| $b$             | 바이어스     |

## ■ 모델 생성

모델 생성은 신경망의 네트워크 구조를 만드는 과정입니다. 입력층의 입력 개수, 출력층의 출력 개수, 중간층의 개수, 층의 종류 등 용도에 맞게 설계합니다.

학습 전에는 뉴런이 올바른 가중치 파라미터와 바이어스를 알지 못하므로 0이나 난수 등으로 초기화합니다. 이 상태의 모델에 테스트 데이터를 주어도 정답을 출력하지 못합니다.

# 딥러닝 단계: 학습

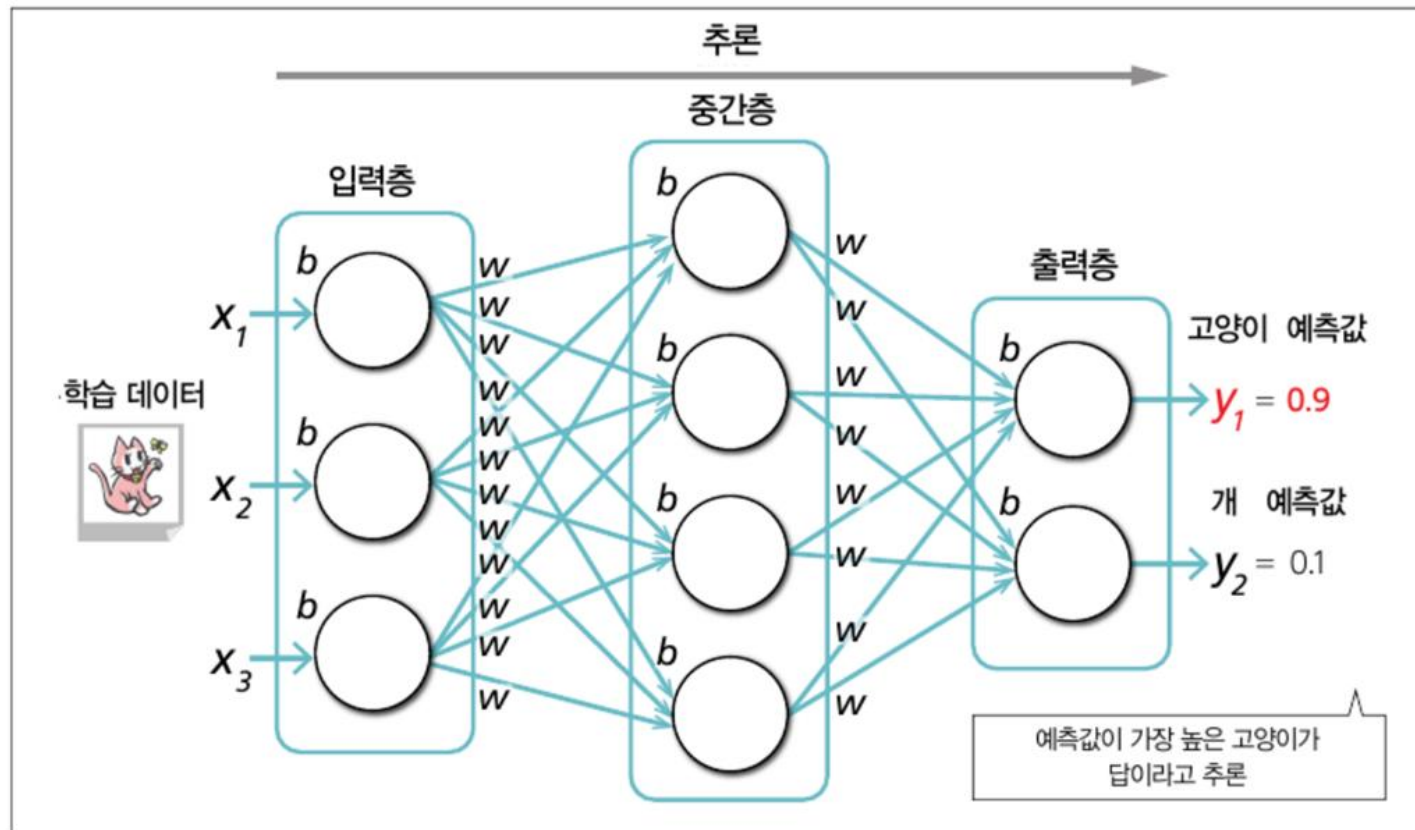


GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍

한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# 답러닝 단계:추론

모델 학습이 완료되면 테스트 데이터를 모델에 입력하여 추론을 수행합니다. 예측값이 가장 높은 것이 답일 것이라고 추론합니다.



GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍

한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# 언어 딥러닝 모델

| 모델명         | 등장연도  |
|-------------|-------|
| RNN         | 1986년 |
| Seq2Seq     | 2014년 |
| Attention   | 2015년 |
| Transformer | 2017년 |
| BERT        | 2018년 |

| 모델명     | 등장연도  |
|---------|-------|
| GPT-2   | 2019년 |
| T5      | 2019년 |
| GPT-3   | 2020년 |
| GPT-3.5 | 2022년 |
| GPT-4   | 2023년 |

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# RNN 모델

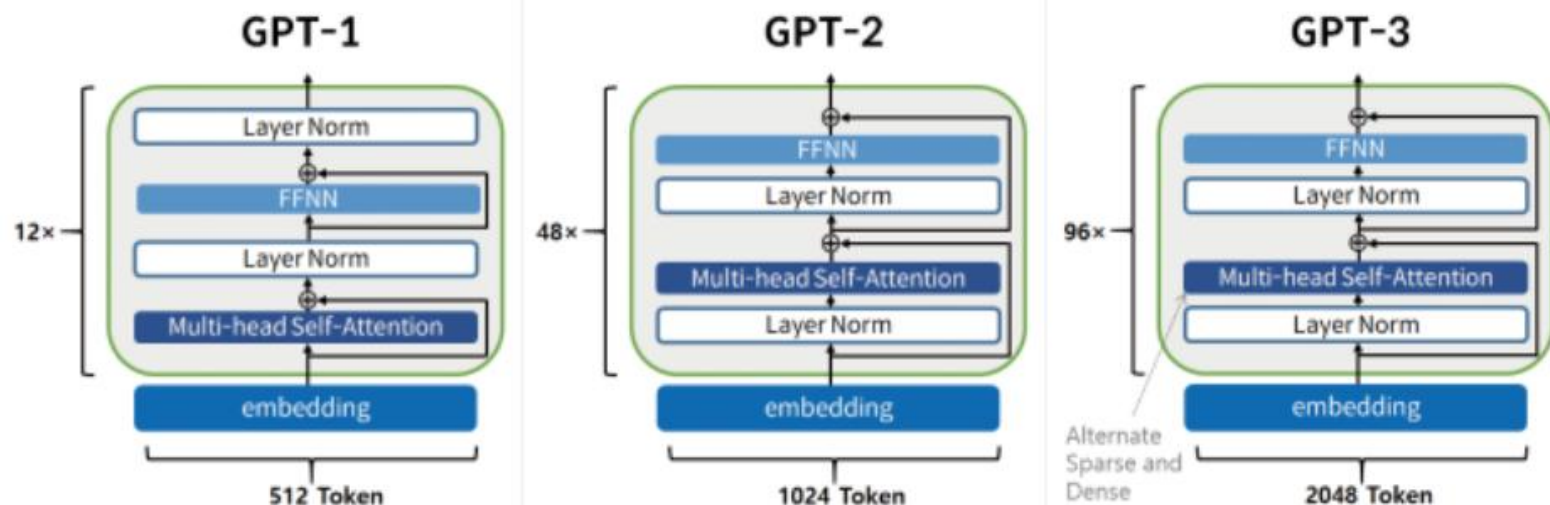
RNN(Recurrent Neural Network)은 입출력으로 계열 데이터를 처리할 수 있는 딥러닝 모델입니다. 주로 자연어 처리, 동영상 분류, 음성 인식에 활용됐으며, Transformer 이전에는 구글 번역과 같은 기계 번역에도 활용됐습니다.

이 모델의 특징은 중간층이 재귀 구조를 갖는다는 점입니다. 예를 들어, 이전 시간대의 중간층 출력을 고려하여 현재 시간대의 중간층 출력을 계산합니다.





# GPT



|                  | GPT-1       | GPT-2 | GPT-3    |
|------------------|-------------|-------|----------|
| 파라미터 개수          | 1억 1,700만 개 | 15억 개 | 1,750억 개 |
| 디코더 블록을 쌓아올린 총 수 | 12개         | 48개   | 96개      |
| 입력 토큰 개수         | 512개        | 1024개 | 2048개    |

GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축



GPT-2는 마치 사람이 쓴 것처럼 자연스러운 문장을 생성할 수 있어 악용이 우려된다는 이유로 논문 발표가 연기되어 화제가 된 Transformer 기반의 딥러닝 모델입니다. Transformer의 decoder 부분만 중첩된 구조의 모델로, 임의의 문장에 이어지는 다음 단어를 예측하는 처리에 강점이 있습니다.

GPT-2를 개발한 OpenAI에서는 다음과 같은 4종류의 사전 학습된 모델을 제공하고 있으며, 파라미터 수가 많을수록 정확도가 높은 것으로 알려져 있습니다. 한국어 사전 학습 모델은 SK 텔레콤에서 40GB 이상의 텍스트로 학습시킨 KoGPT2가 공개돼 있습니다.

- GPT-2(small): 1억 2,400만 파라미터
- GPT-2(medium): 3억 5,500만 파라미터
- GPT-2(large): 7억 7,400만 파라미터
- GPT-2(xl): 13억 파라미터

KoGPT2 / 한국어 GPT-2

<https://github.com/SKT-AI/KoGPT2>

# 자기 회귀 모델

GPT-2의 사전 학습 모델 활용 방법으로는 파인튜닝과 함께 제로샷(Zero Shot) 기법이 제안되고 있습니다. 제로샷 기법은 임의의 문장에 이어지는 다음 문장을 예측하는 능력을 이용해 학습되지 않은 과제를 추론하는 방법입니다.

예를 들어 'Q. 서울N타워의 높이는? A.'에 이어지는 문장을 추론하여 '239m'라는 답변 문장이 나오는 등 학습하지 않은 질문에 대답을 하는 과제를 수행합니다. GPT-2의 성능으로는 아직 제로샷 기법으로 풀 수 있는 것이 제한적이지만, 텍스트 생성의 성능을 향상시키면 어떤 과제라도 풀 수 있는 범용 모델이 될 수 있다는 가능성을 보여주었습니다.

## 자기 회귀 모델

GPT-2와 같이 decoder 부분을 겹겹이 쌓아 올린 구조의 모델을 자기 회귀 모델이라고 합니다. 잘 하는 작업은 텍스트 생성입니다. 자기 회귀 모델로는 다음과 같은 모델이 있습니다.

GPT, GPT-2, GPT-3, CTRL, Transformer-XL, Reformer, XLNet

## KoGPT2란?

KoGPT2는 GPT-2 모델을 파인튜닝한 한국어 언어 모델로, SKT-AI에서 한국어 성능의 한계를 개선하기 위해 개발했습니다.

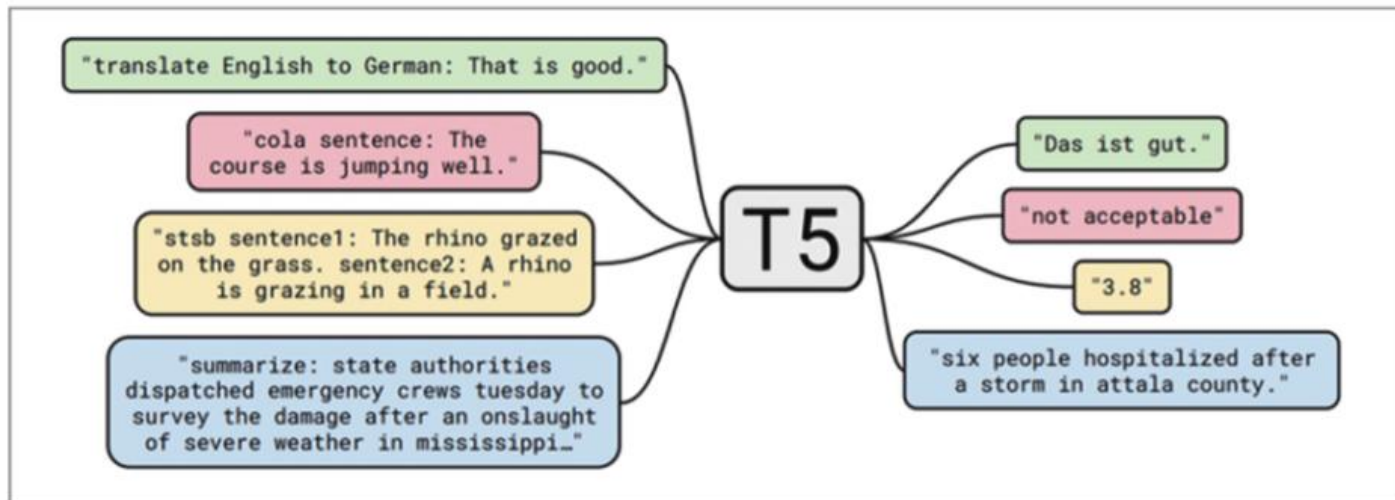
한국어 위키백과 외에 뉴스, 모두의 말뭉치, 청와대 국민 청원 등의 다양한 데이터를 모델의 학습에 사용했으며, 아래 주소에서 데모 버전을 이용해볼 수 있습니다.

### KoGPT2 데모 버전

<https://huggingface.co/spaces/gogamza/kogpt2-base-v2>

T5는 텍스트를 텍스트로 변환하는 Transformer 기반 모델입니다. 학습 데이터를 바꾸는 것만으로 질의응답, 요약, 번역 등 다양한 자연어 처리 작업을 동일한 모델로 해결할 수 있는 것이 특징이며, Transformer의 encoder 부분과 decoder 부분을 모두 사용하는 구조의 모델입니다.

- 질의응답: 입력 '질문' → 출력 '답변'
- 요약: 입력 '원문' → 출력 '요약'
- 번역: 입력 '영어' → 출력 '한국어'



# Sentence-to-Sentence

## Sequence-to-Sequence 모델

T5와 같이 텍스트를 텍스트로 변환하는 모델을 Sequence-to-Sequence 모델이라고 합니다. 잘 하는 작업은 번역, 요약, 질의응답입니다. Sequence-to-Sequence 모델로는 다음과 같은 모델이 있습니다.

BART, Pegasus, MarianMT, T5, MBart

2014년에 발표된 Seq2Seq(Sequence-to-Sequence)와 이름이 같아서 헷갈리지만, 여기서는 Transformer 기반 딥러닝 모델의 분류명으로 Sequence-to-Sequence 모델이라는 명칭을 사용하고 있습니다. 이와는 별도로 텍스트에서 텍스트로 변환하는 모델을 일반적으로 Sequence-to-Sequence(또는 Text-to-Text)라고 부르기도 합니다.

GPT-3는 사람이 쓴 것 같은 자연스러운 문장을 생성할 뿐만 아니라, 자연어부터 프로그램의 소스코드와 웹페이지의 레이아웃까지 생성해 화제를 모았던 GPT-2의 차기 모델입니다. GPT-2에 비해 117배나 더 큰 파라미터를 가지고 있습니다.

- GPT-3: 1,750억 파라미터

GPT-3의 사전 학습 모델 활용 방법으로는 파인튜닝과 함께 프롬프트 프로그래밍이라는 방법이 제안되었습니다. 자연어로 '어떤 지식을 어떻게 사용하고 싶은지' 예시(프롬프트)를 제시함으로써 정확도 향상을 유도합니다. 예를 들어, 오른쪽과 같이 '간단한 덧셈을 하는 프로그램입니다'라는 작업 설

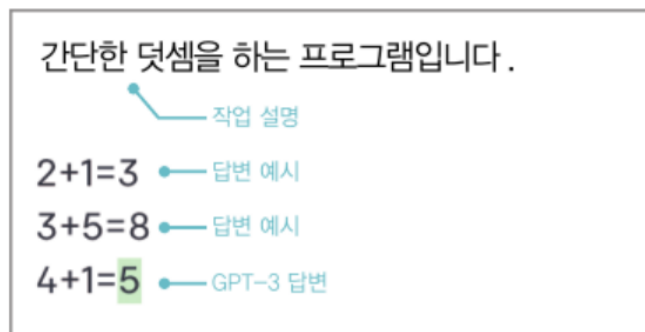


그림 1-3-8 GPT-3의 프롬프트와 답변

명과 함께 '2+1=3', '3+5=8'이라는 답변 예시를 보여주면 '4+1='의 계산 결과를 돌려줍니다. 일반 문자가 사람이 입력한 프롬프트, 녹색이 GPT-3가 생성한 텍스트입니다.

이를 통해 GPT-3는 다양한 자연어 처리 작업을 수행할 수 있게 되었습니다. 그러나 추론 정확도는 실제 수준에는 한 단계 부족한 것 같았습니다.



GPT-3.5는 인간의 피드백을 통한 강화학습을 이용하여 언어 모델과 인간 제시의 정합성을 높인 모델입니다. 강화학습은 기계학습의 기법 중 하나로, 어떤 환경 속에서 행동을 선택하고 그 행동으로 얻을 수 있는 보상을 극대화하도록 학습하는 방법 중 하나입니다. 이를 통해 이전 모델보다 인간의 복잡한 명령을 더 잘 이해할 수 있게 되어 더 높은 품질의 응답을 할 수 있게 되었습니다.

추론의 정확도가 실용적인 수준에 도달하면서 기존 애플리케이션에도 채팅 UI가 탑재되기 시작했고, 라마인덱스, 랭체인 등 LLM으로 애플리케이션을 구축하는 라이브러리도 빠르게 진화하기 시작했습니다.

## 인간의 피드백으로 강화학습을 하는 RLHF

RLHF(Reinforcement Learning from Human Feedback)는 인간의 피드백을 활용하여 강화학습 에이전트(어떤 환경에서 자율적으로 행동하는 프로그램)의 학습을 개선하는 기법입니다. 일반적인 강화학습에서 에이전트는 보상 신호를 받아 행동을 결정하고, 그 결과에 따라 자기 학습을 수행합니다. 하지만 보상 신호가 불분명하거나 목표가 복잡한 경우, 에이전트의 학습이 어려워질 수 있습니다. 이러한 경우 인간이 피드백을 제공함으로써 에이전트의 학습을 도울 수 있습니다. GPT-3.5에서는 RLHF를 통해 인간의 명령을 따르는 능력이 비약적으로 향상되었습니다.

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

GPT-4는 GPT-3.5에서 한 단계 더 진화하여 텍스트 뿐만 아니라 이미지 입력도 가능한 멀티모달 대규모 언어 모델이 되었습니다. 그러나 2023년 5월 현재 이미지 입력 기능은 아직 일반인에게 공개되지 않았습니다.

다양한 벤치마크에서 인간 수준의 성능을 보여주며, 사법시험 모의고사에서는 응시자 중 상위 10% 정도의 점수를 획득할 수 있다는 결과도 보고되었습니다.

파라미터 수는 이전 모델보다 더 많을 것으로 예상되지만, 비공개이므로 알 수 없습니다.



GPT-4 | OpenAI 블로그

<https://openai.com/research/gpt-4>

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축



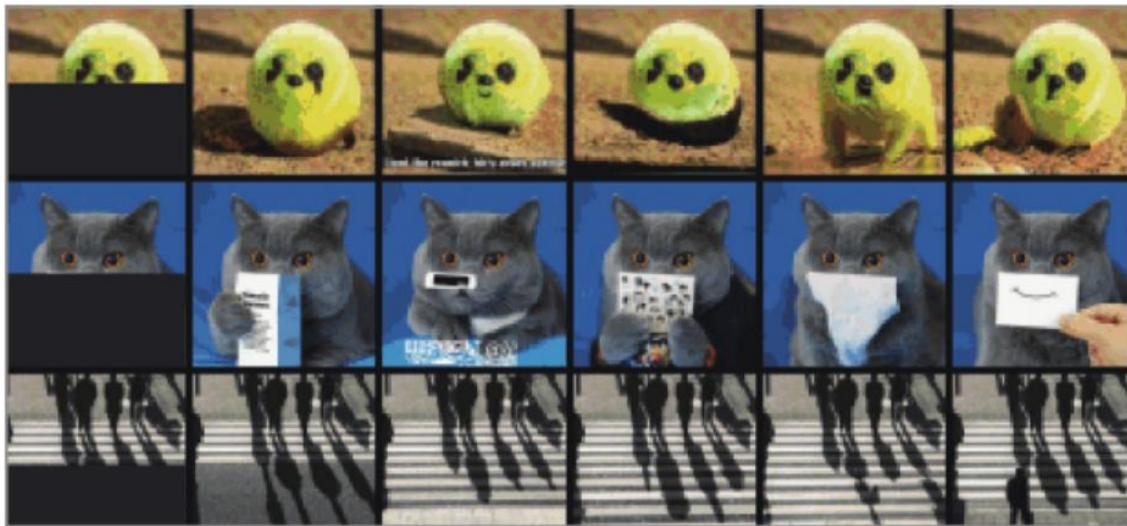
| 특징/차이점  | ChatGPT 3.5                  | ChatGPT 4                                                                                             | ChatGPT 4.o (추정)      |
|---------|------------------------------|-------------------------------------------------------------------------------------------------------|-----------------------|
| 출시일     | 2021년 11월                    | 2023년 3월                                                                                              | 2024년 5월              |
| 모델 크기   | 약 1750억 매개변수                 | 알려지지 않음, 더 큰 모델로 추정                                                                                   | 알려지지 않음, 더 큰 모델로 추정   |
| 성능      | 이전 모델들보다 향상된 성능              | 더욱 개선된 이해력, 추론 능력                                                                                     | 더 높은 이해력, 자연스러운 문장 생성 |
| 다국어 지원  | 가능                           | 가능                                                                                                    | 가능                    |
| 컨텍스트 길이 | 상대적으로 길지만 제한 있음              | 더 긴 컨텍스트 처리 가능                                                                                        | 더 긴 컨텍스트 처리 가능        |
| 응답의 일관성 | 가끔 일관성 문제                    | 일관성 더 높음                                                                                              | 일관성 더 높음              |
| 학습 데이터  | 2021년까지의 데이터                 | 2023년까지의 데이터                                                                                          | 최신 데이터                |
| 특징      | 무난하고 빠른 결과물 생성<br>텍스트 입력만 가능 | 텍스트, 음성, 이미지 입력가능<br>인터넷 검색 with Bing, 고급 데이터 분석, 이미지 생성등 부가 기능 모듈 활용 가능(멀티 모달)<br>GPTs 활용(외부 데이터 접근) | 멀티 모달 기능이 웹브라우저에 합쳐짐  |

# 이미지 생성 트랜스포머

Transformer는 자연어 처리용으로 탄생한 딥러닝 모델이지만, 최근에는 이미지 분류, 이미지 생성 등 이미지 처리에서도 괄목할 만한 성과를 내고 있습니다.

Image GPT는 절반의 이미지에서 나머지 절반을 생성할 수 있는 딥러닝 모델입니다. GPT-2는 임의의 문장 뒤에 이어지는 그럴듯한 문장을 자동으로 생성할 수 있는 모델로 화제가 되었는데, 이 모델의 구조를 이미지에 적용함으로써 반쪽짜리 이미지 뒤에 이어지는 그럴듯한 이미지를 자동으로 생성할 수 있게 됐습니다.

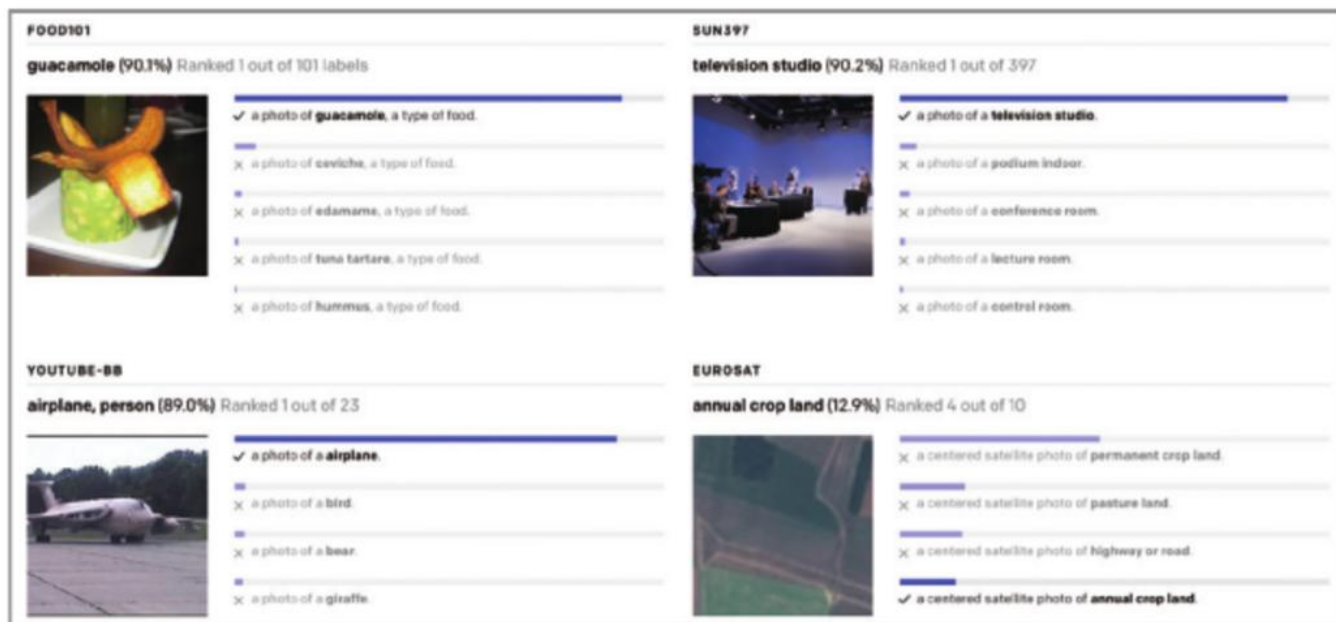
입력 '절반의 이미지' → 출력 '나머지 절반의 이미지'



GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# 이미지 생성 트랜스포머

CLIP은 OpenAI가 개발한 이미지와 텍스트의 연관성을 평가하는 딥러닝 모델입니다. 기존 지도학습의 이미지 분류는 미리 정해진 카테고리만으로 분류할 수 있었던 반면, CLIP은 추론 시 자유롭게 카테고리를 지정하여 분류할 수 있게 되었습니다. GPT-2의 제로샷을 응용한 것입니다.



GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# 이미지 생성 트랜스포머

DALL-E는 텍스트와 이미지 쌍의 데이터 세트를 사용하여 텍스트에서 이미지를 생성하도록 학습된 GPT-3의 120억 파라미터 버전입니다.

입력 '텍스트' → 출력 '이미지'

현실에 존재하는 것뿐만 아니라 동물이나 음식을 의인화하거나, 서로 무관한 것들을 조합해 이미지를 생성하는 것도 가능해졌습니다.

- an illustration of a baby daikon radish in a tutu walking a dog (튜튜 치마를 입고 강아지를 산책시키는 아기 무의 일러스트)



그림 1-3-12 DALL-E가 동물 및 채소를 의인화하여 생성한 이미지 예시

- an armchair in the shape of an avocado (아보카도 모양의 안락 의자)

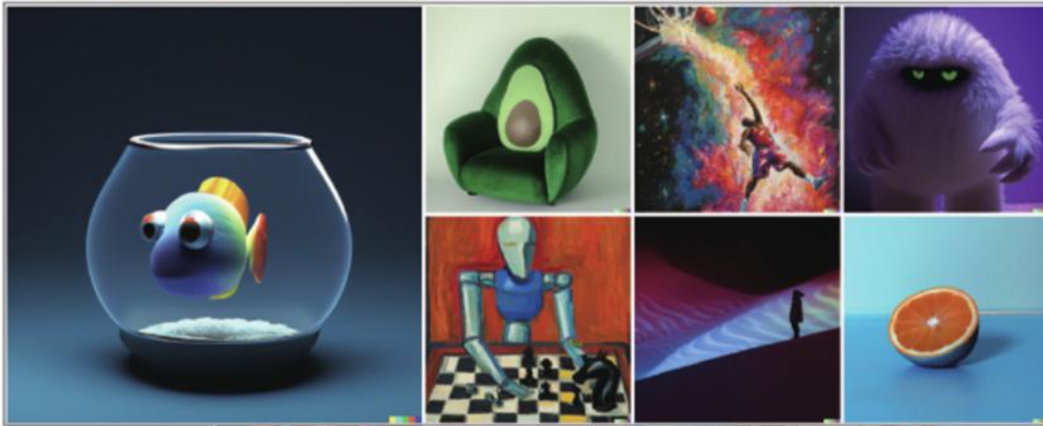


GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# 이미지 생성 트랜스포머

DALL-E 2는 이전 모델보다 해상도를 4배 높여 더욱 사실적이고 정확한 이미지를 생성할 수 있게 되었습니다. 현재 DALL-E라는 이름으로 서비스되고 있는 버전은 이 DALL-E 2입니다.

입력 '텍스트' → 출력 '이미지'



DALL-E 2의 편향성 감소와 안전성 향상 – OpenAI 블로그

<https://openai.com/blog/reducing-bias-and-improving-safety-in-dall-e-2>





## Tacotron2+WaveGlow (2017년)

Tacotron 2는 텍스트를 멜스펙트로그램으로 변환하는 딥러닝 모델이고, WaveGlow는 멜스펙트로그램을 음성으로 변환하는 딥러닝 모델입니다. 이 두 가지를 결합하면 딥러닝 모델로 텍스트를 음성으로 변환할 수 있습니다.

입력 '텍스트' → 출력 '음성'

## NEUTRINO (2020년)

NEUTRINO는 뉴럴 네트워크를 이용한 노래 음성 신디사이저입니다. 가사를 포함한 악보에서 발성 타이밍, 음높이, 성량, 음질, 목소리 굵기 등을 신경망으로 추론하여 목소리를 생성합니다.

인간에 가까운 자연스러운 가창을 구현함과 동시에 악보 작성만으로 생성할 수 있는 간편함까지 더해져 큰 화제를 모으고 있습니다.

입력 '가사가 있는 악보' → 출력 '노래 목소리'

Jukebox는 아티스트, 장르, 가사, 곡의 길이를 지정하기만 하면 자동으로 '노래가 있는 곡'을 생성하는 딥러닝 모델입니다. 소리 데이터뿐만 아니라 해당 곡이 탄생한 연도, 아티스트, 장르 등의 메타데이터와 함께 학습하여 음악의 특징을 학습하고 있습니다.

입력 '아티스트, 장르, 가사, 곡의 길이' → 출력 '가창력이 있는 곡'

## Whisper (2022년)

Whisper는 대화 음성을 텍스트로 변환하는 딥러닝 모델입니다. 한국어를 포함한 다국어 음성 인식, 음성 번역, 언어 인식, 음성 구간 검출 등도 가능한 모델입니다. 오픈소스로 OpenAI에 공개되었습니다.

입력 '대화 음성' → 출력 '텍스트'

Whisper 소개

<https://openai.com/research/whisper>

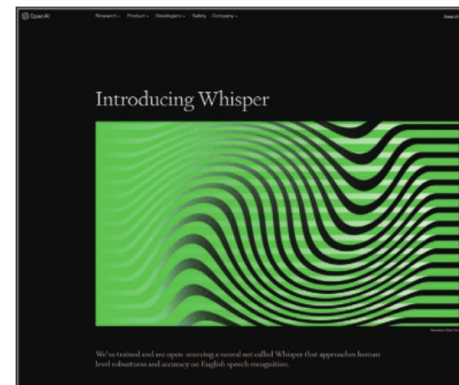


그림 1-3-16 Whisper 웹사이트

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**

한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# Agent

에이전트(agent)는 사용자의 요청에 따라 어떤 기능을 어떤 순서로 실행할지 결정하는 모듈입니다. 체인은 미리 정해진 기능을 수행하지만 에이전트는 사용자의 요청에 따라 수행되는 기능이 달라집니다. 이 에이전트가 수행하는 특정 기능을 도구(tool)라고 합니다.

에이전트는 '행동 이유 추론'과 '이유에 따른 행동'을 번갈아가며 수행합니다. '행동'은 외부 환경에 영향을 주어 새로운 정보를 '관찰'로 수집합니다. '추론'은 외부 환경에 영향을 미치지 않지만, 대신 상황과 맥락을 추론해서 미래의 추론과 행동에 유용한 정보를 업데이트해서 내부 상태에 영향을 미칩니다.





# AI Agent

- 사람의 개입 없이 스스로 작업을 수행하고 의사 결정을 내리는 프로그램
- 자동화 : 반복적이거나 단순한 작업을 자동으로 수행
- 효율성 : 시간과 비용 절감
- 자율성 : 사람이 일일이 지시하지 않아도, 스스로 판단하여 상황에 맞게 작업을 수행

# AI Agent 원리

- ① 입력: 파이썬으로 프로그램 작성
- ② 추론: 파이썬 코드를 실행할 수 있는 도구를 사용할 계획
- ③ 행동: 파이썬 코드를 실행할 수 있는 도구로 프로그램을 작성
- ④ 관찰: 오류가 표시되는 것을 관찰
- ②' 추론: 오류를 수정할 계획을 수립
- ③' 행동: 파이썬 코드를 실행할 수 있는 도구로 프로그램을 수정
- ④' 관찰: 프로그램이 정상적으로 동작하는 것을 관찰
- ⑤ 작업 완료



Reason Only

Act Only



ReAct(Reason+Act)

# AI Agent 적용 사례

- 웹 검색 자동화
  - 웹 검색 결과를 기반으로 답변을 생성합니다. / ‘오늘 날씨 알려줘’ 명령을 실행
- 데이터 분석
  - 데이터를 수집하고 분석합니다. / 엑셀 데이터 분석, 보고서 생성
- 자연어 처리
  - 대화형 AI 서비스에서 자연어를 해석하고 답변합니다. / 챗봇, 고객 서비스 에이전트
- 코드 생성/수정
  - 코드 자동 생성, 수정 작업을 수행합니다. / 깃허브 코파일럿(GitHub Copilot), ChatGPT 코딩
- 다단계 작업 수행
  - 여러 단계의 작업을 자동으로 처리합니다. / 데이터수집 -> 분석 -> 보고서 생성

# LLM, RAG, AI Agent 역할과 차이

| 구분    | LLM                  | RAG                       | AI 에이전트                  |
|-------|----------------------|---------------------------|--------------------------|
| 역할    | 두뇌(Brain)            | 정보 수집(Information Source) | 작업 관리자(Task Manager)     |
| 주요 기능 | 질문 이해, 자연어 처리, 응답 생성 | 외부 데이터베이스 또는 웹에서 정보 검색    | 목표 설정, 계획 생성, 작업 실행, 재계획 |

| 구분     | LLM           | RAG            | AI 에이전트              |
|--------|---------------|----------------|----------------------|
| 핵심 역할  | 텍스트 생성        | 검색+생성          | 작업 계획+실행+재계획         |
| 구성 요소  | 단일 LLM        | LLM+검색 엔진      | LLM+계획+메모리+실행        |
| 정보 출처  | 훈련 데이터만 사용    | 훈련 데이터+검색 데이터  | 훈련 데이터+웹 검색+작업 결과    |
| 실시간 정보 | X             | 웹에서 실시간 검색     | 웹 검색, 작업 실행          |
| 목표 지향성 | 단일 질문에 대한 응답만 | 단일 질문에 대한 응답만  | 사용자 목표에 맞춰 스스로 작업 실행 |
| 대표 예시  | ChatGPT, 클로드  | RAG(LangChain) | AutoGPT, 랭체인 에이전트    |

- LLM 과 LLM + RAG

질문: "2025년 미국 대통령이 누구인가요? 그의 주요 경제 정책 세 가지를 알려주고, 정책의 시장 반응을 조사해주세요."

## LLM

- 2024년 미국 대통령이 누가 될지는 선거 이후에 확정됩니다.
- 도널드 트럼프는 감세, 보호무역주의, 이민 제한을 주요 경제 정책으로 내세웠습니다.
- 조 바이든은 인프라 투자, 최저임금 인상, 증세를 주요 경제 정책으로 내세웠습니다.
- 각 정책에 대한 시장의 반응은 감세, 인프라 투자에 대해선 긍정적일 수 있으나, 증세, 보호무역주의에 대해선 부정적 반응이 예상됩니다.

## LLM+RAG

- 2024년 미국 대통령 선거에서 도널드 트럼프 전 대통령이 당선되었습니다. 주요 경제 정책 세 가지는 다음과 같습니다: 1. 대규모 감세 정책: 법인세를 현행 21%에서 15%로 인하하고, 개인 소득세를 폐지하여 관세로 대체하는 방안을 추진하고 있습니다. 2. 보호무역주의 강화: 모든 수입품에 대해 10%의 관세를 부과하는 등 보호무역주의 정책을 강화하여 자국 산업을 보호하려는 계획입니다. 3. 이민 제한 강화: 불법 이민자 추방을 확대하고, 멕시코 국경 장벽 건설을 완료하는 등 이민 제한 정책을 강화할 예정입니다. 시장 반응은 다음과 같습니다: • 단기적 경제 성장 기대: 감세와 관세 정책으로 인해 단기적으로 경제 성장이 촉진될 것이라는 기대가 있습니다. • 재정 적자 증가 우려: 대규모 감세로 인해 연방 재정 적자가 증가할 것이라는 우려가 제기되고 있습니다. • 인플레이션 상승 가능성: 보호무역주의와 이민 제한 정책으로 인해 노동력 공급이 감소하고, 이에 따라 임금 상승과 인플레이션 압력이 높아질 수 있다는 전망이 있습니다. 이러한 정책들은 미국 경제에 단기적인 활력을 줄 수 있으나, 장기적으로는 재정 적자 증가와 인플레이션 상승 등의 부작용이 우려됩니다.트럼프의 경제 정책과 시장 반응 [https://www.investopedia.com/trump-economic-proposals-presidential-election-8731809?utm\\_source=chatgpt.com](https://www.investopedia.com/trump-economic-proposals-presidential-election-8731809?utm_source=chatgpt.com)[https://www.vox.com/politics/368004/donald-trump-policy-positions-issues-guide?utm\\_source=chatgpt.com](https://www.vox.com/politics/368004/donald-trump-policy-positions-issues-guide?utm_source=chatgpt.com)[https://elpais.com/internacional/elecciones-usa/2024-11-04/compare-las-propuestas-de-los-candidatos-a-la-presidencia-de-estados-unidos-2024.html?utm\\_source=chatgpt.com](https://elpais.com/internacional/elecciones-usa/2024-11-04/compare-las-propuestas-de-los-candidatos-a-la-presidencia-de-estados-unidos-2024.html?utm_source=chatgpt.com)

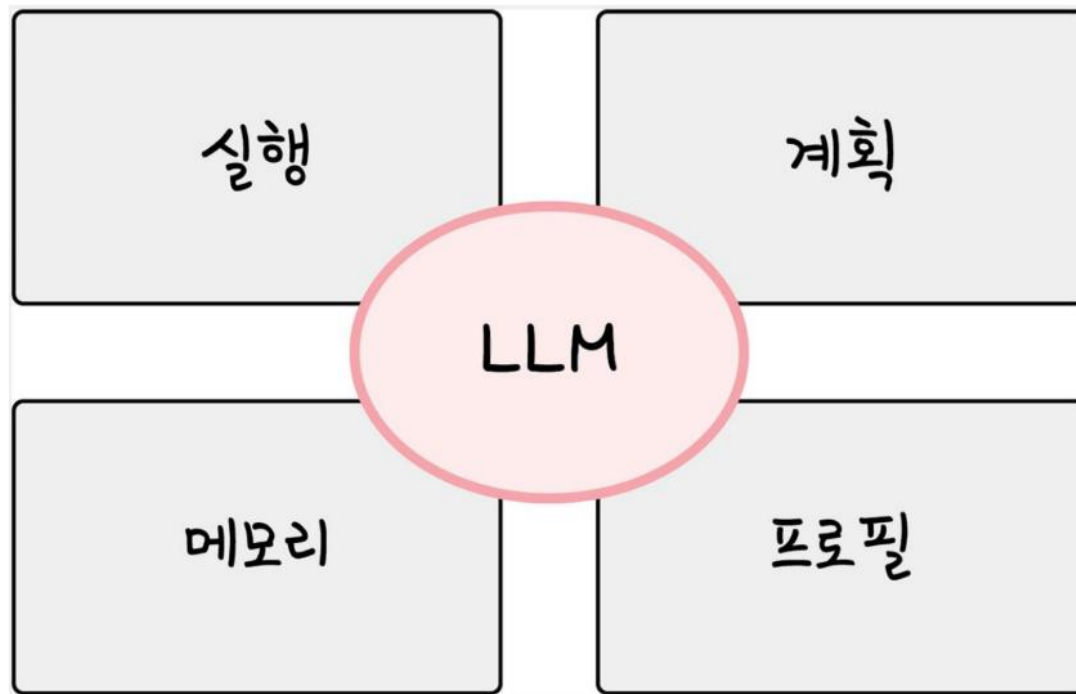
# LLM, RAG, AI Agent 예

- LLM + RAG + AI Agent

| 단계 | 작업 내용                                 | 구성 요소 |
|----|---------------------------------------|-------|
| 1  | 사용자 명령을 수신합니다                         | 에이전트  |
| 2  | LLM을 활용해 계획을 세웁니다                     | 에이전트  |
| 3  | 검색 시스템을 통해 '2024년 미국 대통령' 정보를 수집합니다   | RAG   |
| 4  | RAG가 반환한 정보를 LLM에게 전달합니다              | 에이전트  |
| 5  | 수집된 정보를 요약하여 '도널드 트럼프가 대통령이다'라고 응답합니다 | LLM   |
| 6  | 결과가 충분하지 않으면 재계획을 실행합니다               | 에이전트  |
| 7  | 사용자가 볼 수 있는 최종 응답을 생성합니다              | LLM   |

하위 작업으로 문제를 나누면 복잡한 문제를 단순화하고, 병렬로 처리할 수 있으며, 실패한 작업만 재계획하고, 에이전트를 전문화하고, 성능을 개선할 수 있음

# AI Agent 구성요소와 동작 방식

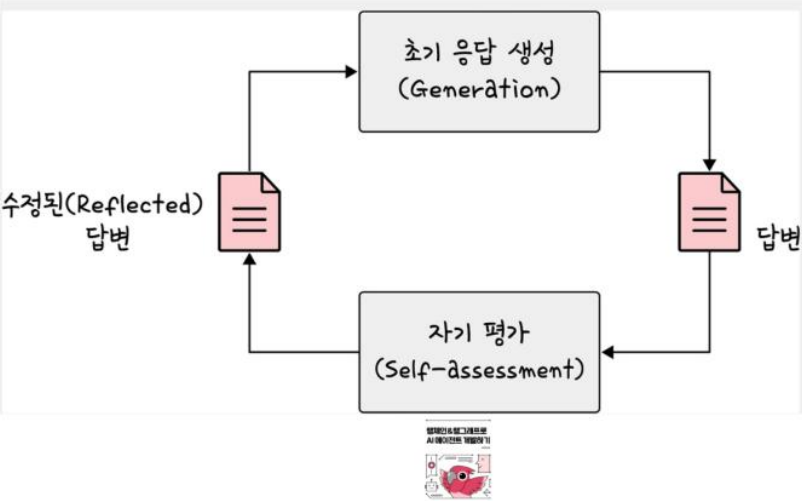


사용자 명령 ↔ 계획 (Planning) ↔ 실행 (Action) ↔ 작업 결과 평가 ↔ 재계획 (필요 시) ↔ 최종 응답 생성 ↔ 종료



# AI 패턴 : 반응 패턴

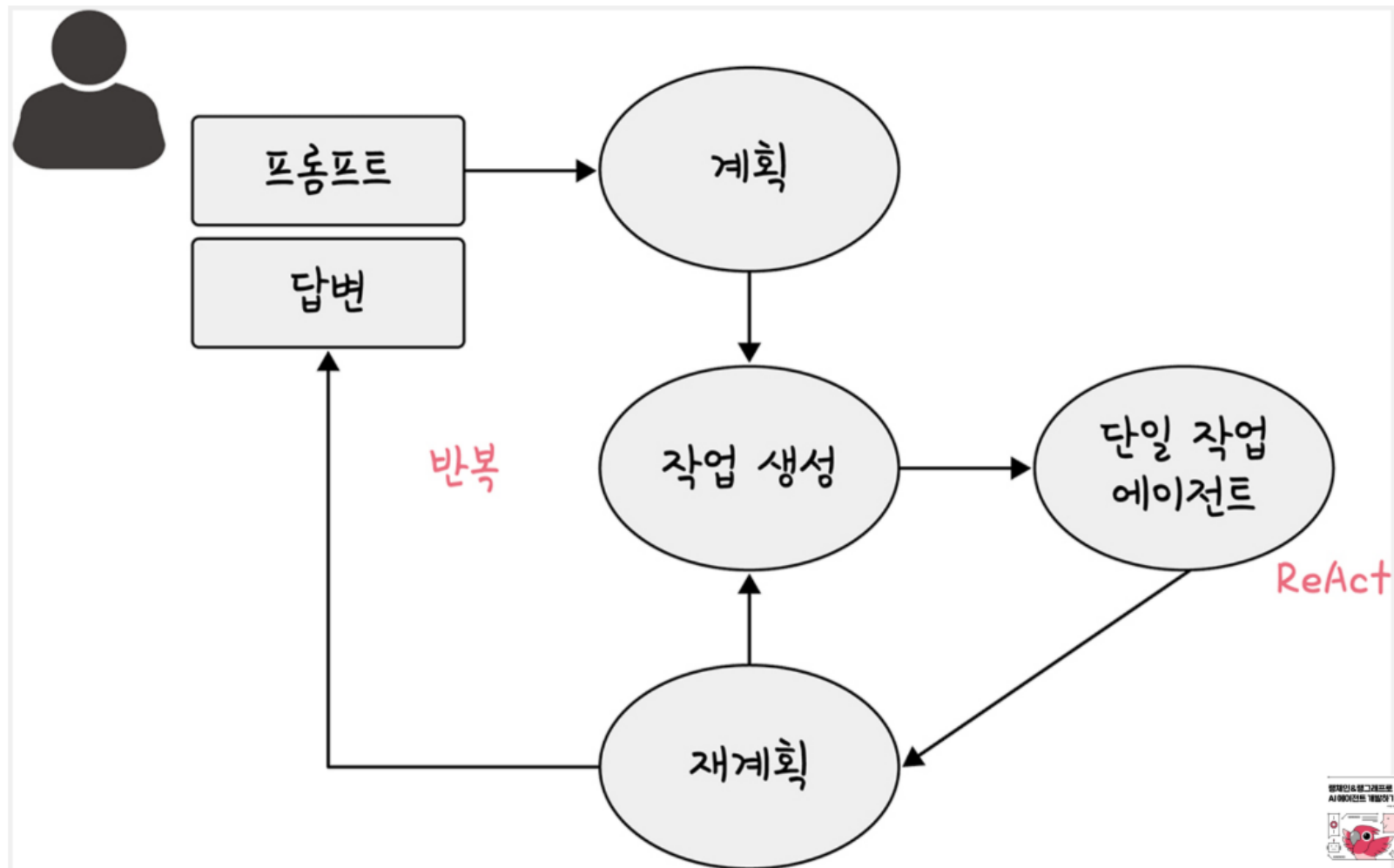
- 문서 생성 AI
- 교육 플랫폼 AI



| 장점         | 설명                                    |
|------------|---------------------------------------|
| 품질 향상      | 단순한 1회 출력이 아닌, 여러 번의 피드백을 통해 품질 향상    |
| 정확성 개선     | AI가 직접 오류를 수정하므로 정확한 결과를 반환할 가능성이 높아짐 |
| 복잡한 문제 해결  | 모호한 표현을 명확하게 하거나 추가 설명을 제공할 수 있음      |
| 개발자의 개입 감소 | 개발자의 피드백 없이도 AI가 스스로 평가하고 개선할 수 있음    |

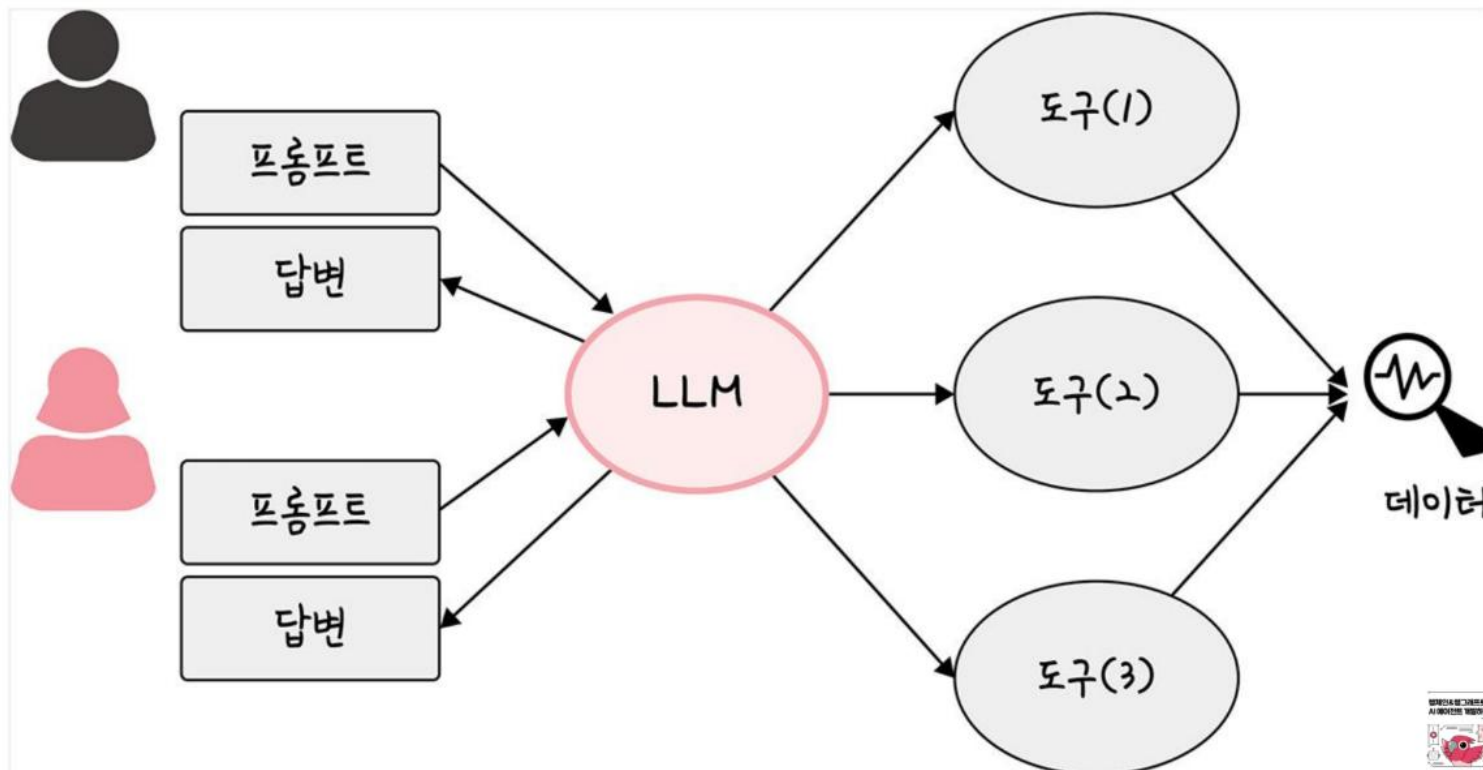
| 단점         | 설명                                         |
|------------|--------------------------------------------|
| 계산 비용 증가   | 여러 번의 피드백이 발생하므로 추가 비용과 처리 시간이 필요          |
| 복잡성 증가     | 단순한 응답 생성에 비해 더 많은 메모리와 작업 단계가 필요          |
| 무한 루프의 가능성 | AI가 계속해서 출력을 개선하려고 할 경우, 무한 반복에 빠질 가능성이 있음 |
| 정지 조건 필요   | 특정 종료 조건(품질 기준, 반복 횟수 제한 등)이 필요            |

# AI 패턴 : 계획 패턴



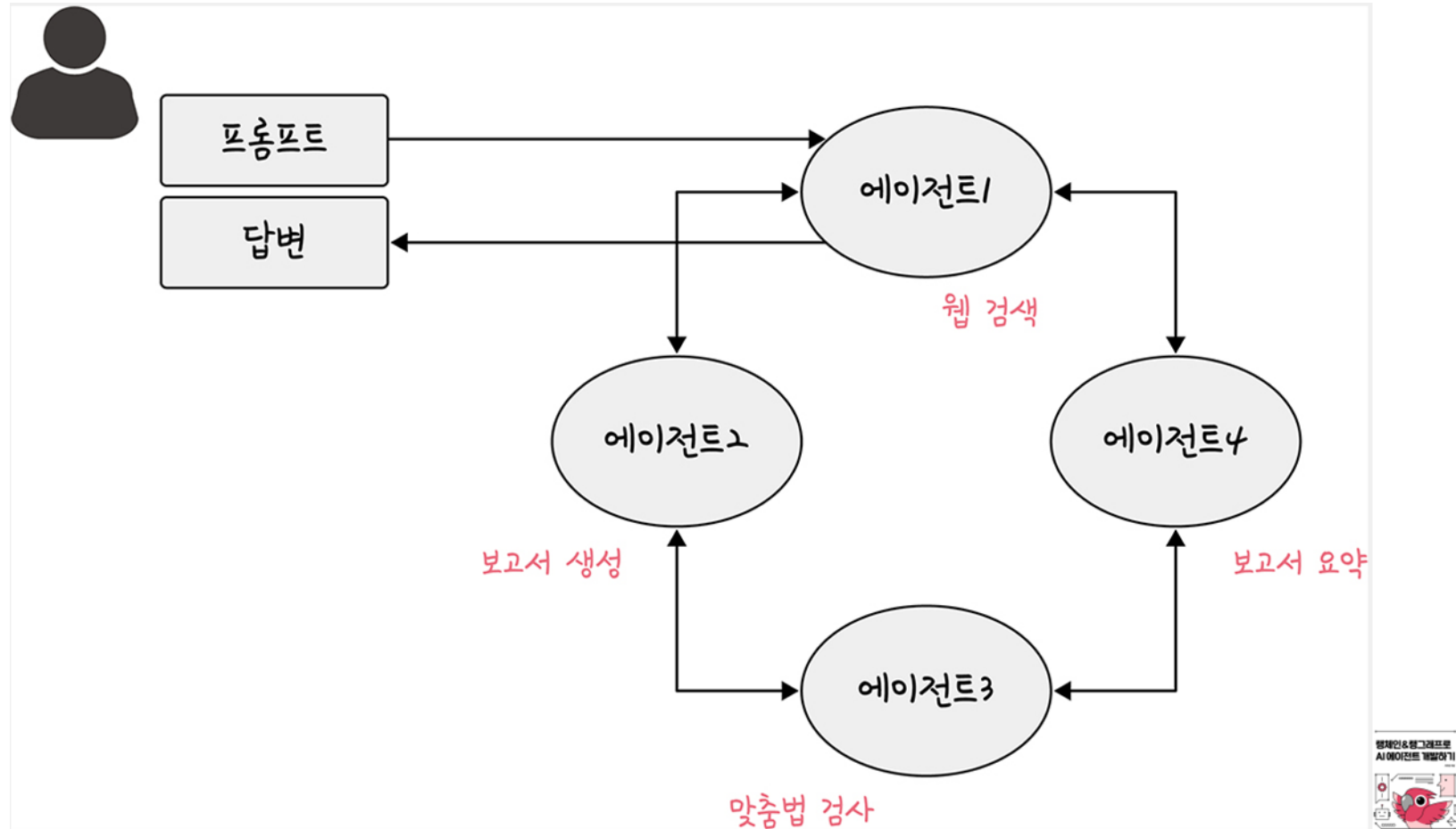
# AI 패턴 : 도구 사용 패턴

- 하나의 거대한 LLM 이 모든 작업을 처리하는 대신, 전문화된 도구에 특정 작업을 분배함으로써 더 효율적이고 확장 가능한 AI 시스템을 구축



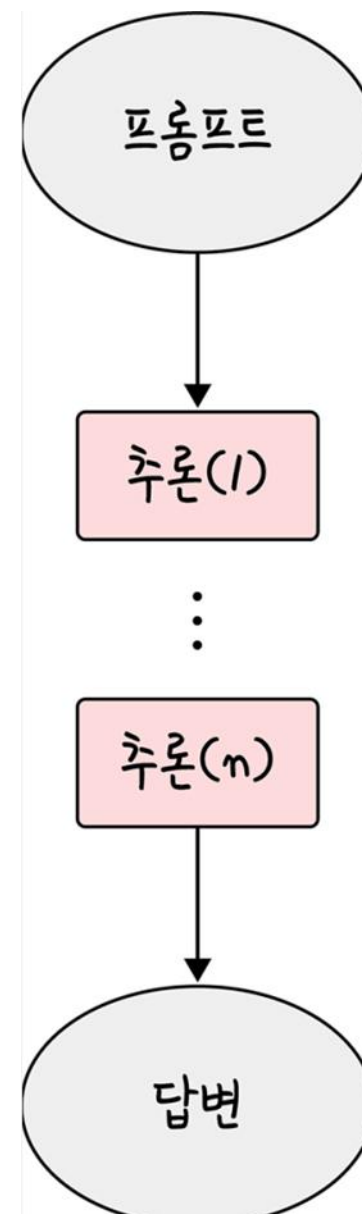
# AI 패턴 : 멀티에이전트 패턴

- 콘텐츠 제작과 같이 다양한 전문 지식이 요구되는 작업에 효과적



# AI 패턴 : CoT 프롬프팅 패턴

- AI가 문제를 단계별로 사고(Thinking)하고 추론하도록 유도하는 기법
- 일반적으로 AI는 질문을 받으면 즉각적으로 답변하려고 하지만, CoT는 AI가 답변을 내리기 전에 논리적 사고의 단계를 거치도록 유도하기 때문에 비교적 정확한 답변을 얻을 수 있음
- 수학 문제 해결, 법률 분석등 활용



# OpenAI API 이해

- OpenAI API 기본 구조 및 사용법

## OpenAI란?

OpenAI는 영리법인 OpenAI LP와 그 모회사인 비영리법인 OpenAI Inc.로 구성된 인공지능 연구소입니다. 2015년 말에 샘 알트만과 일론 머스크 등이 샌프란시스코에서 설립했습니다. 인류 전체에 도움이 되는 방식으로 친근한 인공지능을 보급하고 발전시키는 것을 목표로 삼고 있습니다.

OpenAI

<https://openai.com/>

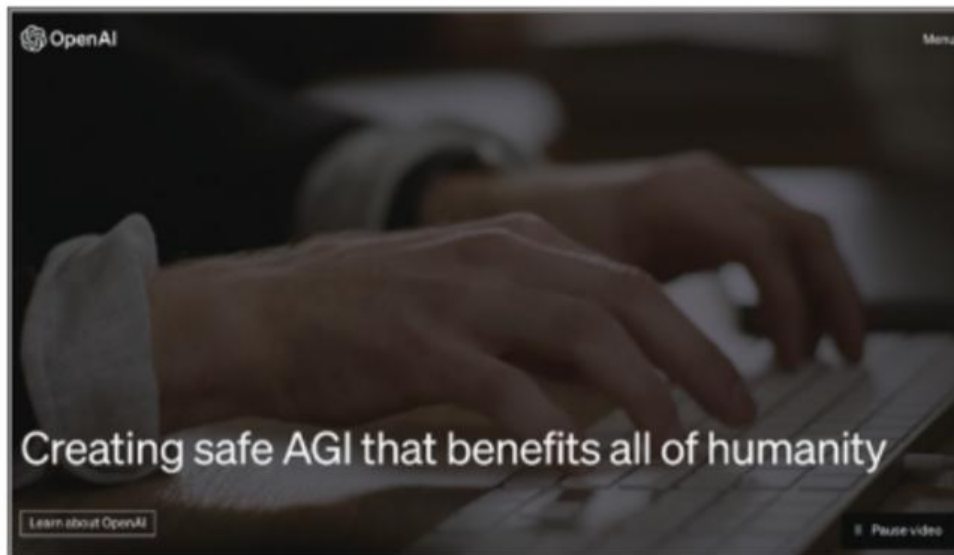


그림 OpenAI의 이념

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

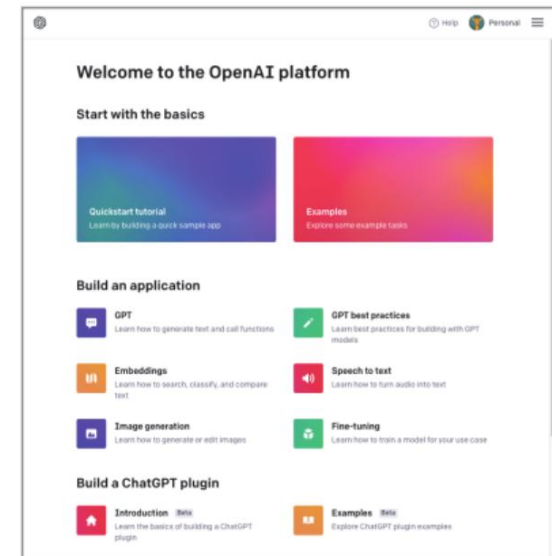


OpenAI API

<https://platform.openai.com/>

## OpenAI API란?

OpenAI API는 OpenAI가 제공하는 자연어 처리 API(Application Programming Interface)입니다. API란 소프트웨어 애플리케이션끼리 서로 정보를 주고받기 위한 인터페이스로, 개발자가 자신의 애플리케이션에 GPT-4나 GPT-3.5의 기능을 통합하는 데 사용할 수 있습니다.



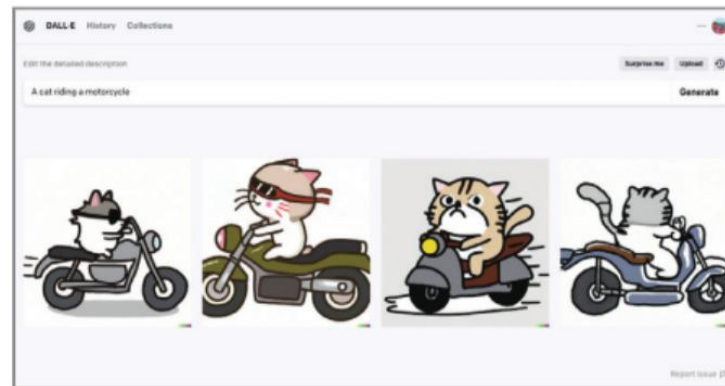
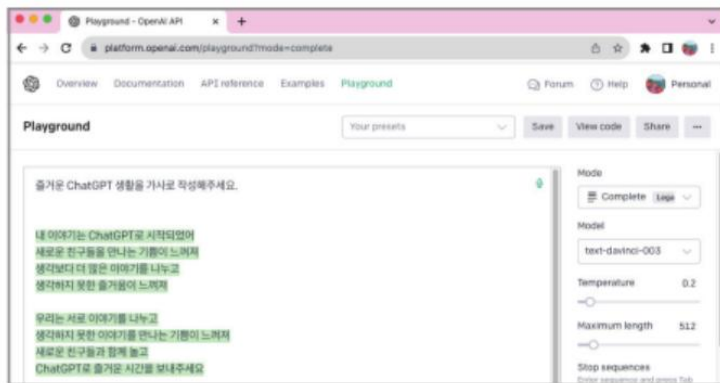
- 텍스트 생성
- 이미지 생성
- 임베딩 생성
- 파인튜닝
- 모델링
- 음성 텍스트 변환(STT)

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한편으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

## OpenAI의 플레이그라운드와 DALL-E의 웹 UI

OpenAI API는 기본적으로 파이썬과 같은 프로그래밍 언어를 사용해야 하지만, 웹에서 이용할 수도 있습니다. 웹 UI(Web UI, Web User Interface)는 웹 페이지에서 조작할 수 있는 UI로, OpenAI API로 어떤 일을 할 수 있는지 간단하게 테스트해 볼 수 있습니다.

OpenAI API 사이트에서는 OpenAI API의 자연어 처리를 실행할 수 있는 OpenAI 플레이그라운드(Playground)와 이미지 생성을 실행할 수 있는 DALL-E의 웹 UI를 제공하고 있습니다.



**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한편으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# OpenAI 출시 모델

## OpenAI의 제품 출시 연표

OpenAI는 2020년 OpenAI API를 통해 GPT-3를 제공한 이후 다양한 제품을 출시해 왔습니다. 다음은 이를 정리한 연표입니다.

표 OpenAI 제품 출시 연표

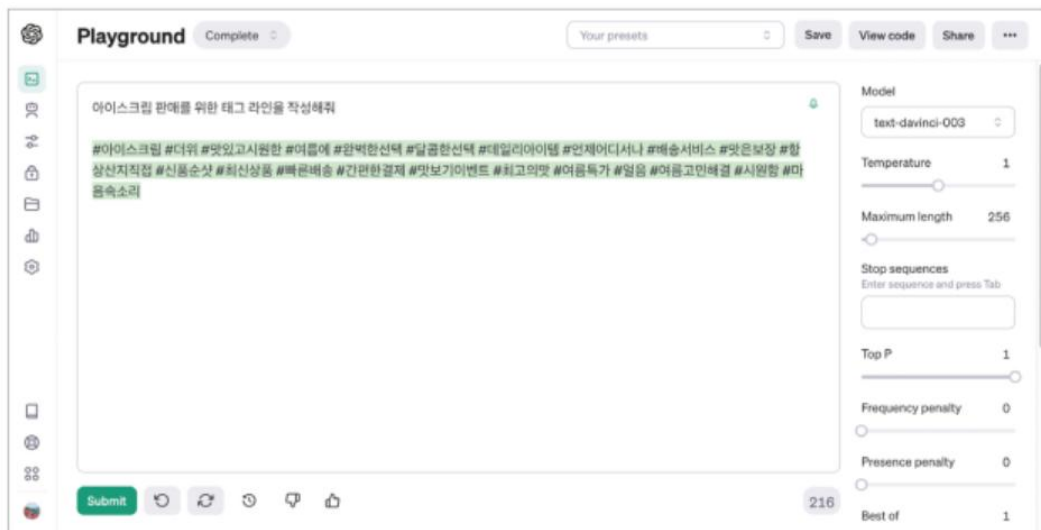
| 년     | 월일      | 제품명                        |
|-------|---------|----------------------------|
| 2020년 | 6월 11일  | OpenAI API, GPT-3, 플레이그라운드 |
|       | 8월 10일  | Codex                      |
| 2021년 | 1월 5일   | DALL-E (발표만), CLIP         |
|       | 6월 29일  | Github Copilot             |
| 2022년 | 4월 13일  | DALL-E 2 (발표만)             |
|       | 7월 20일  | DALL-E (클로즈 베타)            |
|       | 9월 21일  | Whisper                    |
|       | 9월 28일  | DALL-E (공개)                |
|       | 11월 3일  | DALL-E API                 |
|       | 11월 29일 | GPT-3.5 (text-davinci-003) |
|       | 11월 30일 | ChatGPT                    |
|       | 12월 25일 | text-embedding-ada-002     |
| 2023년 | 2월 1일   | ChatGPT Plus               |
|       | 3월 1일   | ChatGPT API, Whisper API   |
|       | 3월 9일   | Azure OpenAI Service       |
|       | 3월 14일  | GPT-4                      |
|       | 3월 23일  | ChatGPT Plugins (클로즈 베타)   |
|       | 3월 23일  | Git Hub Copilot X (클로즈 베타) |

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# OpenAI 의 다른 모델 사용하기

- OpenAI 플레이그라운드: <https://platform.openai.com/playground?mode=complete>

플레이그라운드는 챗봇이라기보다는 자동 완성 툴처럼 생각하면 됩니다. 일기를 작성할 수도 있고, 질문을 던질 수도 있습니다. 사용 방법은 프롬프트에 입력한 후 하단의 [Submit] 버튼을 클릭하면 됩니다.



Model  
gpt-3.5-turbo

**text-davinci-003**  
Most capable model in the GPT-3 series. Can perform any task the other GPT-3 models can, often with higher quality, longer output and better instruction-following. It can process up to 4,000 tokens per request.  
**STRENGTHS**  
Complex intent, cause and effect, creative generation, search, summarization for audience

**GPT-3**  
text-davinci-003  
text-curie-001  
text-babbage-001  
text-ada-001  
[Show more models](#)  
**CHAT**  
gpt-4-vision-preview  
gpt-4-1106-preview

**GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍**  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축

# ChatGpt의 오해와 한계

1. 실시간 학습 능력 (없음)
2. 알고 있는 지식의 기간 (학습이 끝난 시간)
3. 정보를 검색하지 않는다 (정보검색 엔진과 협업)
4. 숫자에 약합니다 (개선)
5. 거짓말 주의 ((Hallucination, 할루시네이션)(개선)
6. 이미지를 그리는 인공지능 (이미지 생성 모델과의 협업)
7. 사고하는 능력 (없음)
8. 한글과 영어에 성능 차이 (개선)

# 자연어 처리

## 언어 및 자연어 처리

자연어는 한국어, 영어 등 우리가 일상적으로 사용하는 언어를 말하며, 파이썬, C 언어 등 컴퓨터에서 사용하는 언어인 프로그래밍 언어와 구별하기 위해 이렇게 부릅니다.

자연어를 컴퓨터로 처리하는 일련의 기술을 자연어 처리라고 하는데, 구어에서 문어까지 자연어가 갖는 의미를 다양한 방법으로 분석합니다.

GPT-4, GPT-3.5는 자연어뿐만 아니라 프로그래밍 언어도 학습하지만, 주요 용도는 자연어 분야에서의 활용이므로 GPT-4, GPT-3.5 기술도 자연어 처리라고 부르고 있습니다.

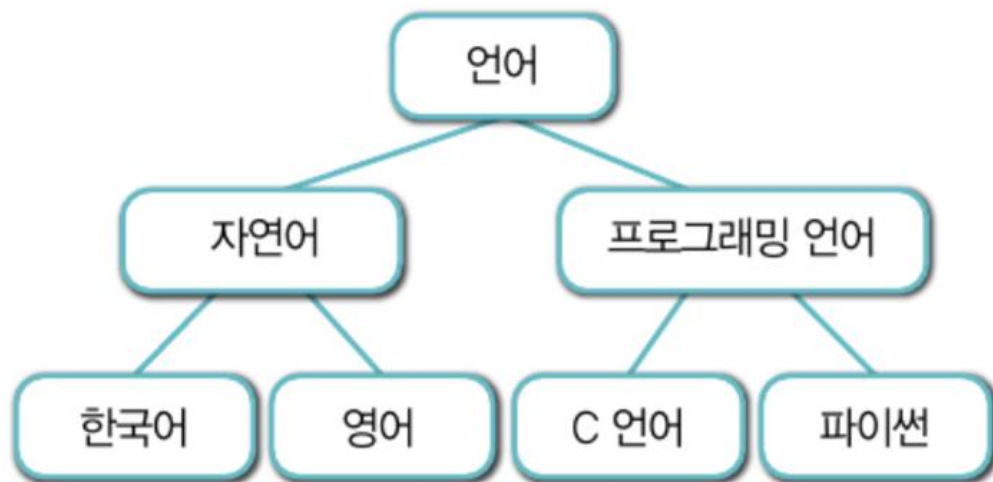


그림 자연어와 프로그래밍 언어

## ❖ LLM이란?

- 정의: 대규모 언어 모델(Large Language Model)은 인간 언어를 이해하고, 생성하고, 상호 작용하도록 설계된 고급 AI 시스템입니다. 이는 방대한 양의 텍스트 데이터와 복잡한 알고리즘을 사용하여 구축되었습니다.
- 기능: 이 모델은 데이터로부터 언어 패턴을 학습합니다. 이 데이터를 처리함으로써 텍스트 작성, 질문 답변, 대화 참여 등의 작업을 수행할 수 있습니다.
- 작동 방식: LLM은 규칙을 사용하여 명시적으로 프로그래밍하는 대신 예제를 통해 '학습'하는 기계 학습이라는 방법을 사용합니다. 이는 앞에 나오는 단어가 제공하는 문맥을 기반으로 단어 또는 단어 시퀀스의 가능성을 예측합니다.
- 간단한 비유: LLM을 엄청난 양의 책을 읽고 요청을 받았을 때 읽은 내용을 바탕으로 정보를 기억하고 구성할 수 있는 매우 세심한 학생이라고 생각하십시오.

# LLM 특징

## 자연스러운 문장 생성

LLM은 임의의 문장 뒤에 오는 다음 문장을 예측하는 능력을 학습하고 있습니다. 임의의 문장을 입력하면 이어지는 문장을 예측하여 출력하는 딥러닝 모델입니다. 이때 입력 문장은 프롬프트(prompt), 출력 문장은 컴플리션(completion)이라고 부릅니다.

예를 들어 LLM에 '나는'을 입력하면 '나는 고양이'와 같은 문장을 출력합니다.

LLM은 문장 생성뿐만 아니라 다양한 자연어 처리 작업에 활용할 수 있습니다. 예를 들어, LLM에 '대한민국에서 가장 높은 산은?'이라는 질문을 입력하면 '한라산'과 같은 응답을 출력합니다.



그림 1-1-8 LLM(대규모 언어 모델)의 입출력

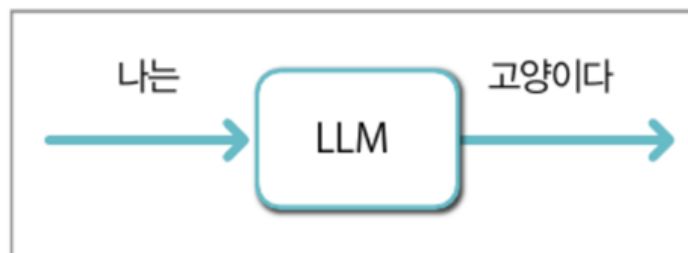
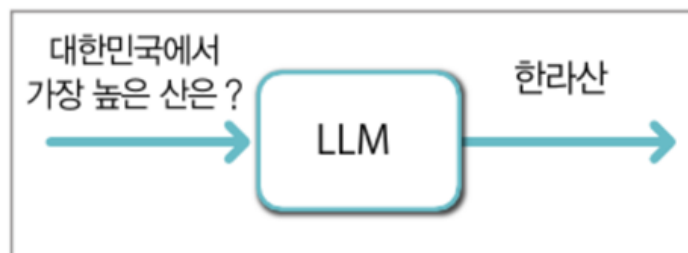


그림 1-1-9 문장 생성의 입출력



GPT-4, ChatGPT, 라마인덱스, 랭체인을 활용한 인공지능 프로그래밍  
한권으로 끝내는 OpenAI API 기반 LLM 애플리케이션 구축



## 논리적 사고력 향상

그림 1-1-10 질의응답의 입출력

LLM은 임의의 문장에 이어 다음 문장을 예측하는 모델이기 때문에 자연스러운 문장을 만드는 데 능숙하지만, 수학처럼 논리적인 사고를 조합하는 데에는 서투른 것으로 알려져 있습니다. 하지만 질문의 마지막에 '단계별로 생각해 보자(Let's think step by step)'라는 문구를 덧붙이면 문제의 정답률이 높아진다는 것을 발견했습니다.

예를 들어, 다음과 같은 질문을 하면 자연스러운 문장을 반환하지만, 계산 결과가 잘못됐다는 것을 알 수 있습니다.

### [입력]

16개의 공이 있다. 그중 절반은 골프공이고, 골프공의 절반은 파란색이다. 파란색 골프공은 몇 개일까?

### [출력]

2개입니다. → 오답

질문의 마지막에 '단계별로 생각해 보자'를 추가하면 단계별로 추론하여 정답을 도출할 수 있습니다.

## [입력]

16개의 공이 있다. 그중 절반은 골프공이고, 골프공의 절반은 파란색이다. 파란색 골프공은 몇 개일까?

단계별로 생각해 보자.

## [출력]

주어진 정보에 따르면 공의 총 수는 16개입니다.

그 중 절반은 골프공이므로 골프공의 개수는  $16 / 2 = 8$ 개입니다. 또한, 골프공의 절반은 파란색이므로 파란색 골프공의 개수도  $8 / 2 = 4$ 개입니다.

따라서 파란색 골프공은 4개입니다. → 정답

이처럼 단계적으로 추론을 유도하여 복잡한 문제의 추론 정확도를 높이는 방법을 생각의 사슬 (Chain of Thought, CoT)이라고 부릅니다.

# 지침의 명확성

## ❖ 신속한 엔지니어링을 위한 명확성 정의:

명확성(Clarity)은 쉽게 이해되고 모호함이 없는 상태를 말합니다.

ChatGPT 프롬프트에 대한 신속한 엔지니어링의 맥락에서 명확성은 지침이 원하는 작업이나 질문을 모델에 효과적으로 전달하도록 보장합니다.

## ❖ 명확한 지침의 예:

"소설 '앵무새 죽이기'의 주요 사건을 요약해 보세요."

"엽록소의 역할을 포함하여 식물의 광합성 과정을 설명합니다."

"실제 사례를 사용하여 경제학의 수요와 공급 개념을 설명합니다."

## ❖ 모호한 지침의 예:

"책에서는 무슨 일이 벌어지나요?" (모호함 - 어떤 특정 정보를 요청하는지 불분명함)

"화학을 설명해보세요." (모호함 - 너무 광범위하고 구체성이 부족함)

"상황을 설명하세요." (모호함 - 맥락이나 방향이 부족함)

명확한 지침은 ChatGPT에 예상되는 내용에 대한 구체적인 지침을 제공하여 정확하고 관련성이 높은 응답을 촉진합니다. 반면에 모호한 지시는 오해나 관련 없는 결과를 초래할 수 있습니다.

### ❖ 프롬프트에서 컨텍스트 제공의 중요성:

컨텍스트는 작업이나 질문에 대한 ChatGPT의 이해를 향상시킵니다.

필요한 배경 정보를 제공하고 보다 관련성이 높은 응답을 위한 장면을 설정합니다.

컨텍스트는 ChatGPT의 출력이 일관되고 사용자 기대와 일치하는지 확인하는 데 도움이 됩니다.

### ❖ 컨텍스트가 포함된 프롬프트의 예:

"당신이 작은 마을에서 발생한 살인 사건을 수사하는 형사라고 상상해 보십시오. 범죄 현장과 찾은 증거를 설명하십시오."

"당신은 새로 발견된 행성을 탐험하는 과학자입니다. 행성의 대기, 지리, 생명 가능성에 대해 자세히 설명하는 보고서를 작성하세요."

"당신은 유명 손님을 위한 특별한 저녁 식사를 준비하는 셰프입니다. 당신의 요리 전문성을 보여주는 메뉴를 디자인하세요."

### ❖ 컨텍스트가 없는 프롬프트의 예:

"범죄 현장을 묘사하세요." (컨텍스트 없음)

"새로운 행성에 대해 글을 써보세요." (컨텍스트 없음)

"메뉴를 생성하세요." (컨텍스트 없음)

# 프롬프트 길이

## ❖ 최적의 프롬프트 길이:

프롬프트의 길이는 부담을 주지 않으면서 ChatGPT에 필요한 컨텍스트와 지침을 제공하기에 충분해야 합니다.

이상적으로 프롬프트는 간결하면서도 유익해야 모델이 작업이나 질문을 명확하게 이해할 수 있어야 합니다.

## ❖ 간결한 프롬프트의 예:

"자연의 아름다움에 대한 시를 써보세요."

"양조 과정에서 발효 과정을 설명합니다."

"인공지능의 개념을 설명해주세요."

## ❖ 긴 프롬프트의 예:

"당신이 중세 시대에 도착한 미래의 시간 여행자라고 상상해 보십시오. 그 시대의 사람 및 기술과 상호 작용한 경험을 설명하십시오."

"당신은 외딴 마을에서 발생한 일련의 미스터리한 실종 사건을 조사하는 저널리스트입니다. 지역 주민과의 인터뷰 및 증거 분석을 포함하여 조사 결과를 자세히 설명하는 기사를 작성하세요."

"고대 문명을 전문으로 하는 역사가로서, 당신은 유물이 가득한 숨겨진 무덤을 발견했습니다. 무덤과 그 내용물을 설명하고, 그것을 건설한 문명에 대해 추측해 보세요."

프롬프트를 간결하게 유지하면 ChatGPT가 현재 작업을 빠르게 파악할 수 있으며, 프롬프트가 길면 더 자세한 지침을 제공하거나 모델이 탐색할 복잡한 시나리오를 설정할 수 있습니다. 그러나 충분한 정보를 제공하는 것과 불필요한 장황함을 피하는 것 사이에서 균형을 맞추는 것이 중요합니다.

# 언어 스타일

프롬프트와 예상 응답의 언어 스타일을 일치시키세요.

프롬프트와 예상 응답 간의 언어 스타일의 일관성은 ChatGPT 출력의 일관성과 관련성을 보장합니다.

언어 스타일은 상황, 청중, 원하는 대화 톤에 따라 달라질 수 있습니다.

## ❖ 공식 프롬프트의 예:

"세계화가 경제에 미치는 영향에 대한 자세한 분석을 부탁드립니다."

"기후 변화에 대한 우려를 표현하는 공식 편지를 편집자에게 작성하세요."

"재생 가능 에너지원의 이점과 과제를 설명하는 연구 논문을 작성하십시오."

## ❖ 비공식 프롬프트의 예:

"가장 좋아하는 영화와 그 영화를 좋아하는 이유를 알려주세요."

"꿈의 휴가 목적지와 그곳에서 어떤 활동을 할 것인지 설명하세요."

"항상 당신을 웃게 만드는 재미있는 이야기나 농담을 공유해 보세요."

프롬프트의 언어 스타일을 예상 응답과 일치시키면 ChatGPT에 대한 적절한 기대치가 설정되고 원하는 어조와 격식 수준으로 출력을 안내하는 데 도움이 됩니다.

# 예시 및 형식

## ❖ 예제 및 형식 지정 지침의 중요성:

예시는 명확한 기대치를 제공하고 원하는 응답 형식이나 구조를 보여줍니다.  
형식 지정 지침은 ChatGPT 출력의 일관성과 가독성을 보장합니다.

예:

"2차 방정식을 푸는 단계를 나열하십시오.

계수와 상수를 식별합니다.

2차 공식을 적용합니다:  $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$

xxx를 단순화하고 풀어보세요.

솔루션이 정확한지 확인하세요."

"주말 계획을 논의하는 두 캐릭터 간의 대화를 작성하세요.

등장인물 A : "이번 주말에 뭐 하고 싶어요?"

캐릭터 B : "산행을 하려고 했는데 당신은 어때요?"

캐릭터 A : "좋네요! 참여하겠습니다."

B캐릭터 : "멋져요! 토요일 오전 9시에 등산로 입구에서 만나요."

## 형식화된 프롬프트 예

❖ **프롬프트:** "당신이 새로 개업한 레스토랑에 대한 리뷰를 작성하는 여행 블로거라고 상상해 보십시오. 레스토랑의 분위기, 제공되는 메뉴 및 전반적인 식사 경험에 대한 자세한 설명을 제공하십시오. 리뷰를 향상하려면 고품질 이미지를 포함하십시오."

❖ **형식화된 프롬프트:**

Title: Restaurant Review

Location: [Name of Restaurant], [City]

Introduction: [Brief introduction to the restaurant and its concept.]

Ambiance: [Description of the restaurant's decor, atmosphere, and any unique features.]

Menu Offerings: [Overview of the menu items, including specialties, beverages, and desserts.]

Dining Experience: [Personal impressions of the dining experience, service quality, and overall satisfaction.]

Image Attachments: [Insert high-quality images showcasing the restaurant's interior, dishes, and presentation.]

Conclusion: [Final thoughts and recommendations for readers.]



## ❖ 자연어 이해:

ChatGPT는 대화식 방식으로 사용자 입력을 이해하고 응답합니다.

대화 전반에 걸쳐 맥락을 유지하여 일관되고 관련성이 높은 응답을 가능하게 합니다.

# ChatGPT 애플리케이션

## 1. 콘텐츠 생성:

기사, 이야기, 시 및 기타 창의적인 콘텐츠를 생성합니다.

이메일 초안, 보고서 또는 에세이 작성과 같은 작문 작업을 지원합니다.

## 2. 가상 비서:

정보를 제공하고, 질문에 답변하고, 작업을 지원하는 강력한 가상 비서

## 3. 고객 지원:

문의 사항을 처리하고 문제를 해결할 수 있는 AI 기반 챗봇으로 고객 지원 서비스를 강화합니다.

## 4. 언어 번역:

언어 간 텍스트를 번역하여 다양한 언어 배경의 의사소통을 촉진합니다.

# LLM 한계

LLM이 우리의 삶에 큰 변화를 가져온, 혁신적인 서비스인 것은 사실이나 LLM은 아직 시작하는 단계



## LLM 한계: 편향과 공정성

만약 LLM이 주로 남성 엔지니어의 데이터를 학습했다면, '엔지니어' 라는 단어에 대해 남성 이미지를 더 자주 연상시키는 문장을 생성할 수 있음

실제로 아마존은 직원 채용 시 'AI 면접' 과정이 있었지만 남성에 더 우호적으로 채점하는 것으로 밝혀지면서 AI 면접을 폐지했다고 함

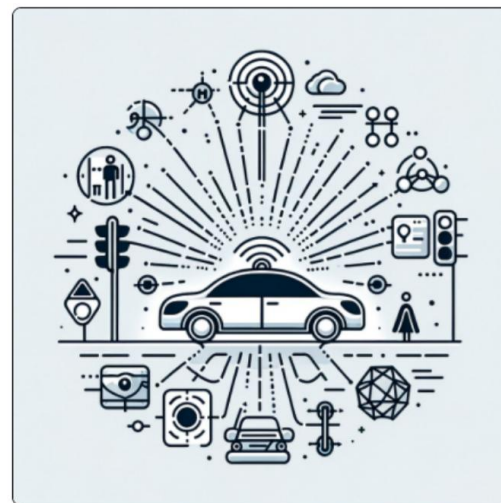
## LLM 한계: 투명성

LLM이 어떤 질문에 대해 특정 대답을 하는 경우 왜 그런 대답을 했는지 그 이유를 사용자에게 설명하는 능력이 부족할 수 있음

분명 학습한 데이터를 기반으로 답변을 한 것은 맞지만 그렇다고 LLM이 학습한 데이터를 그대로 사용자에게 보여주는 것은 아님

LLM 자체적으로 어느 정도 가공을 하는데 그 가공 과정을 인간은 이해할 수 없음

예를 들어 자율 주행 자동차는 복잡한 딥러닝 알고리즘을 사용하여 주변 환경을 인식하고 결정을 내리는데 이때 결정이란 언제 브레이크를 밟을지, 언제 우회전할지 등등 결정하는 것을 의미. 그러나 이러한 결정 과정은 종종 우리 인간이 이해할 수 없는 경우가 많은데, 이러한 문제로 차량이 특정 상황에서 어떻게 행동할지 예측하기 어렵다는 점이 문제임



# LLM 한계: 데이터 의존성과 정보의 일반화

데이터 의존성 : LLM은 특정 언어, 특정 분야(예: 문학)에 한정되어서 데이터를 학습하지 않기 때문에 지금과 같이 다양한 질문에도 답변할 수 있음

하지만 모델이 특정 국가의 소설로만 학습을 할 경우 다른 지역의 문화적 맥락을 반영한 텍스트를 생성하는 데 한계가 있을 수 있음

예를 들어 모델이 오직 프랑스 소설로만 학습된 이 모델은 프랑스 문학, 문화, 역사 및 사회적 맥락에 대해 매우 풍부한 지식을 갖고 있겠지만 모델에 한국의 문화적 배경을 가진 이야기를 생성하도록 요청했다면 우리나라에 대한 충분한 지식이 없기 때문에 우리의 전통, 관습, 역사적 배경 등을 적절히 반영하지 못할 것임

정보의 일반화: ‘데이터 의존성’ 과 반대되는 것으로 LLM이 너무 다양한 데이터를 학습했기 때문에 특정 산업(예: 여행)에 특화된 질문을 할 경우 정밀한 답변을 얻지 못할 수 있음

## LLM 한계: ...

오류 가능성: LLM이 잘못된 정보를 기반으로 문서를 작성할 수 있으며 이는 가짜 뉴스의 확산과 같은 문제를 야기할 수 있음

개인정보 보호: 학습 데이터에 포함된 개인정보(예: 메신저 대화 내용에 포함된 이름, 위치, 개인적 선호 등)를 모델이 학습하고, 이를 생성 과정에서 노출시킬 수 있음

새로운 정보의 결여: 모델이 2023년까지의 데이터로 훈련되었다면, 그 이후에 발생한 사건이나 정보에 대해서는 알지 못하고 과거의 정보를 기반으로 응답할 수 있음

기업 내 데이터 미활용: 기업에서 LLM을 그대로 사용하지 못하는 이유는, 앞의 이유들보다는 LLM을 그대로 기업에서 활용할 경우 기업이 가진 데이터를 활용할 수 없기 때문

# API(Application Programming Interface)

OpenAI는 2023년 3월 1일부터 ChatGPT에 기반한 GPT-3.5-turbo 모델의 API 서비스 시작

API는 별도의 코드를 활용해 ChatGPT 플랫폼을 사용할 수 있게 해주는 기능

API를 활용하면 웹사이트에 접속하지 않고도 본인만의 자동화 코드에서 ChatGPT 서버로 입력을 보내고 결과를 받을 수 있음

API를 통해 다양한 프롬프트 입력 및 결과 생성이 가능하며, 나아가 자신만의 애플리케이션에 LLM를 통합 가능



# OpenAI에서 지원하는 모델

<https://platform.openai.com/docs/models>

<https://openai.com/api/pricing/>

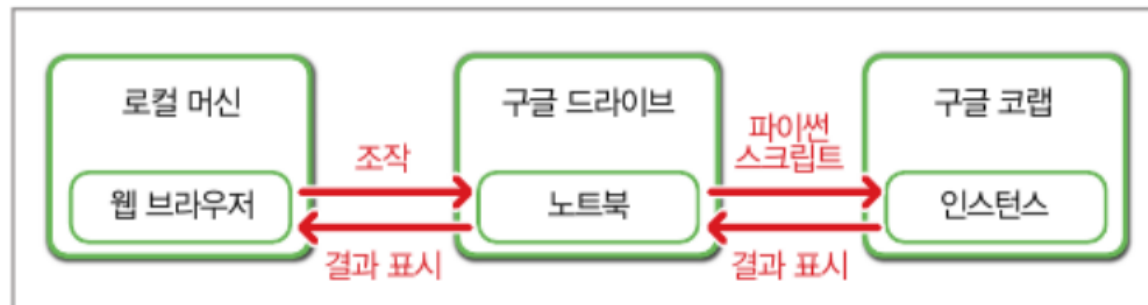
# 랭체인 활용 업무자동화

- 랭체인(LangChain) 프레임워크 소개
- 랭체인(LangChain)의 도구 사용 에이전트 구현

Google Colab(코랩)은 Google에서 제공하는 무료 클라우드 기반 Jupyter 노트북 환경

## ❖ 주요 특징:

1. **무료 사용:** Google 계정만 있으면 무료로 사용할 수 있으며, 기본적으로는 CPU가 제공되지만, GPU와 TPU도 일정 기간 동안 무료로 사용 가능
2. **Jupyter 노트북 기반:** Jupyter 노트북 환경과 유사하게 작동하며, 코드 셀과 마크다운 셀을 사용하여 코드를 실행하고 텍스트를 입력할 수 있음
3. **GPU 및 TPU 지원:** 머신러닝 작업을 위한 GPU와 TPU 자원을 제공
4. **구글 드라이브 연동:** Google Drive와 통합되어 있어, 사용자 데이터 파일을 불러오고 저장할 수 있음
5. **라이브러리 사전 설치:** Colab에는 다양한 Python 라이브러리가 이미 설치되어 있어, 바로 사용 가능

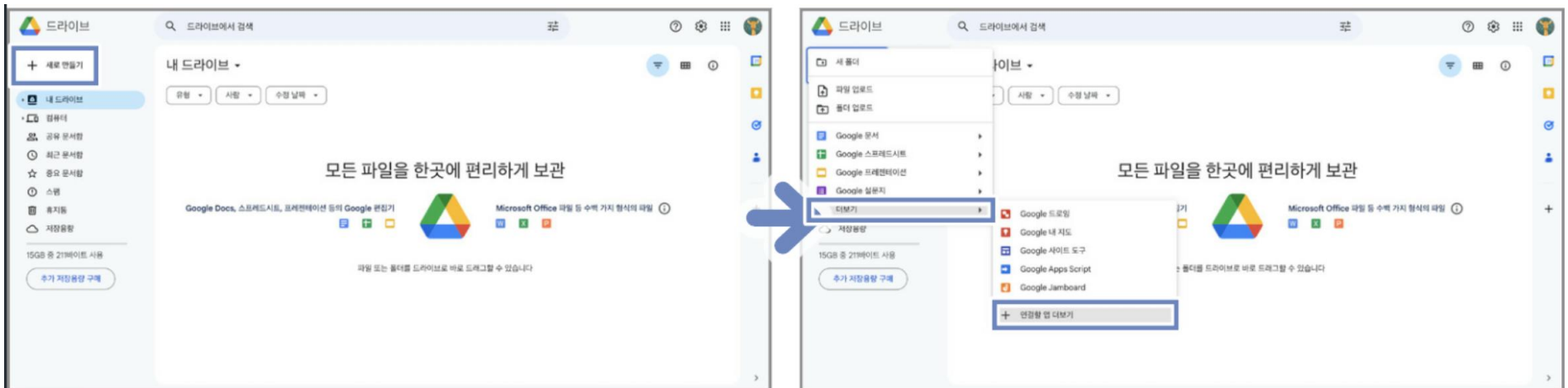


# 코랩 설치

구글 계정 만들기

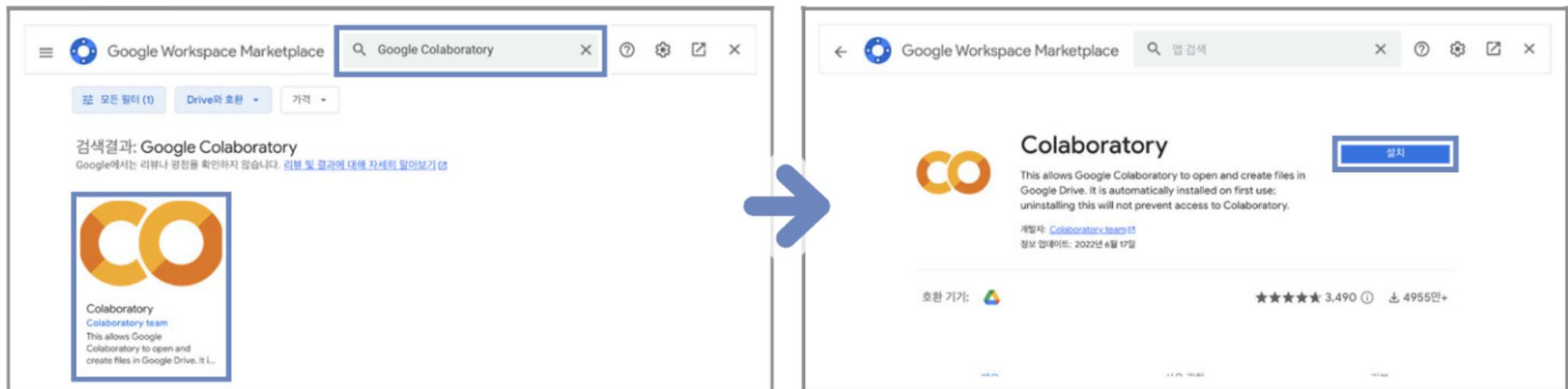
드라이브로 들어오기

신규 → 더보기 → 연결할 앱 더보기



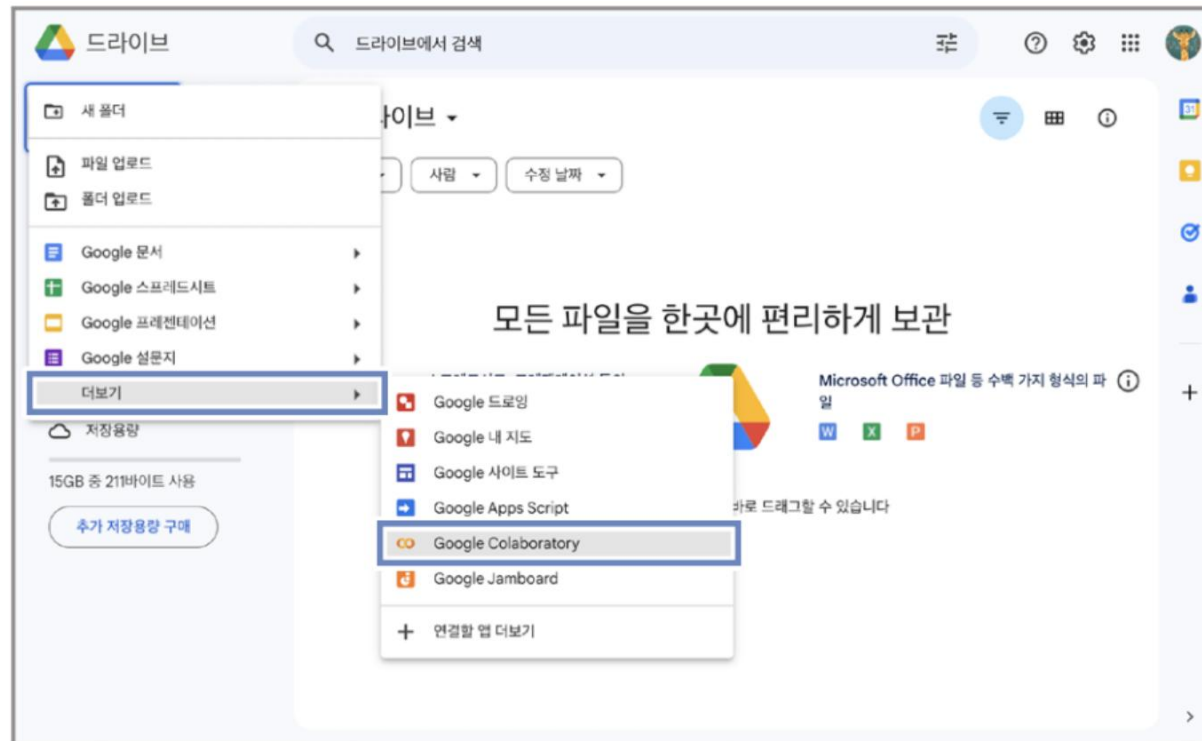
# 코랩 설치

앱 검색 → collaboratory 검색 → 설치 클릭



# 코랩 설치

신규 → 더보기 → colaboratory



# 코랩 툴바



# 코랩 툴바

새로고침

세션 저장소에

업로드

드라이브

마운트

파일





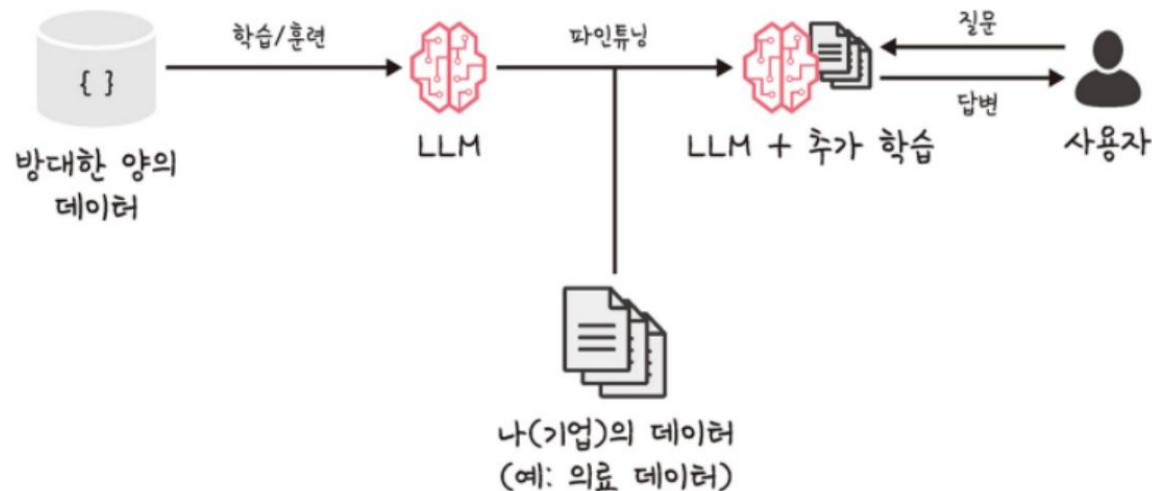
# 파인튜닝(Fine-Tuning)

기존의 모델을 특정한 작업이나 상황에 맞게 조금 더 훈련시키는 과정

이미 많은 것을 배운 LLM을 특별한 상황에 더 잘 맞게 ‘가르치는’ 것

예를 들어 언어를 처리하는 LLM이 이미 많은 책과 글을 읽어서 여러 가지 언어를 이해하는 법을 배웠다면 이 모델을 의학 분야에서 전문적으로 사용하고 싶다면 의학 관련 문서를 추가로 학습시킴

이렇게 하면 이미 배운 기본 지식을 유지하면서 특정 분야에 더 정확하고 효과적으로 대응할 수 있게 됨



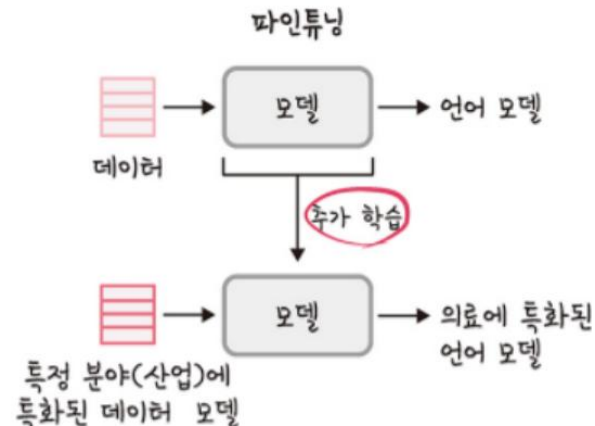
# 파인튜닝

한 분야에서 배운 지식을 다른 분야의 문제 해결에 사용하는 방법을 전이 학습(Transfer Learning)이라고 함

이전 모델에서 학습한 내용이 그대로 다른 작업에도 적용될 수 있도록 정보가 전이되는 형태



※ 추가 학습(파인튜닝)은 필요에 따라 진행



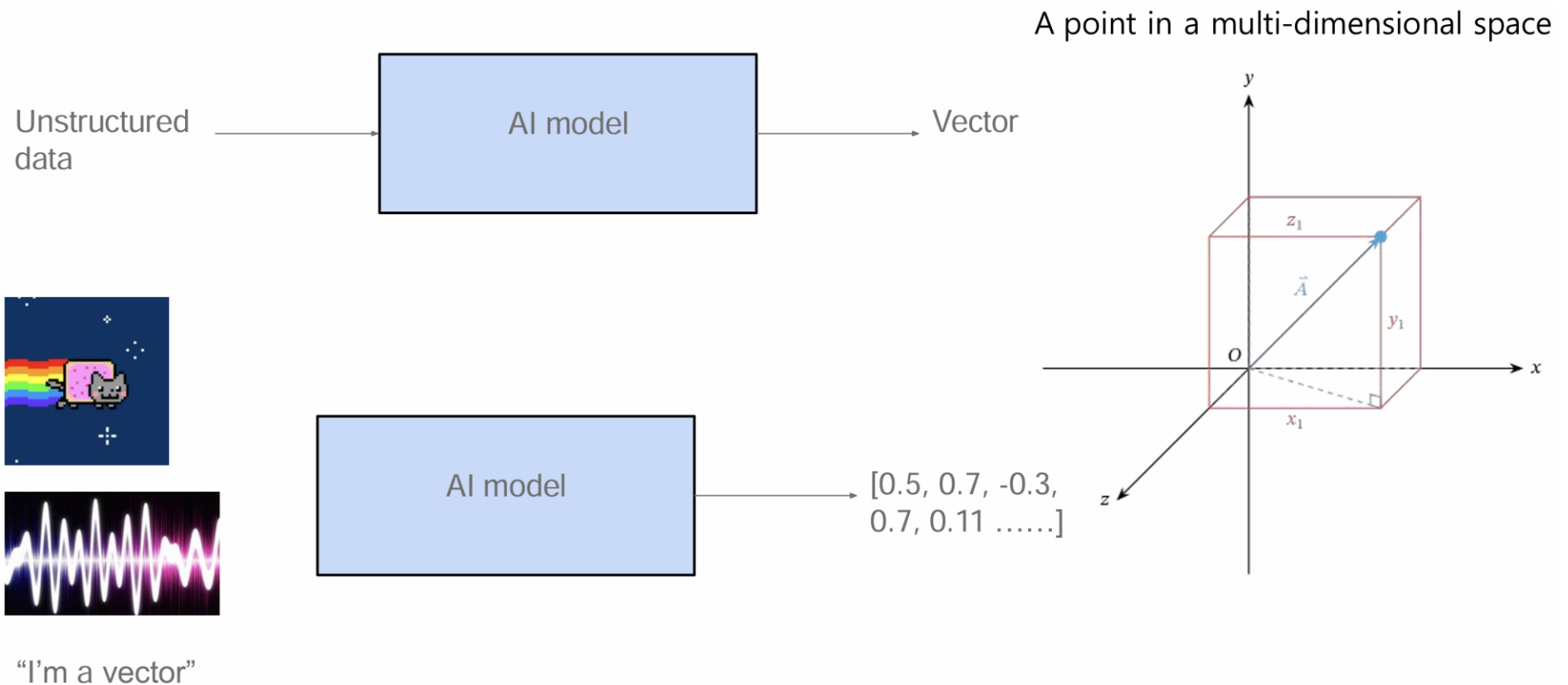
## 파인튜닝의 문제점

이미 만들어진 LLM에 추가 학습만 진행하면 된다고 하지만 어느 정도의 데이터로 어느 정도의 훈련을 시켜야 기대하는 성능(정확도)을 얻을 수 있을지 확신할 수 없음

학습을 위해 과도한 비용이 발생: LLM은 말 그대로 거대 언어 모델이기 때문에 여기에 추가 학습을 시키기 위해서는 클라우드 자원을 활용하는 비용이 과도하게 높음

데이터 준비가 어려움: 파인튜닝을 위해서는 데이터를 '질문-답변' 세트 형식으로 준비해야 하는데, 기존에 보유한 데이터가 이런 형식으로 정리되어 있지 않은 경우가 대부분

# Everything can be a vector



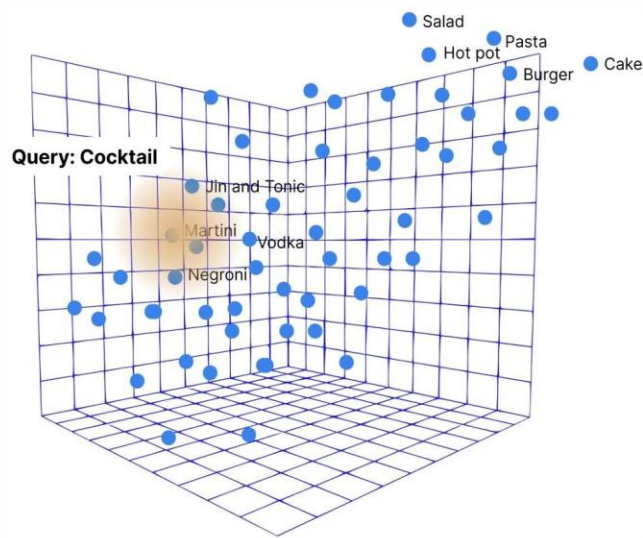
# Embedding 기술의 이해

## ❖ 정의

- Embedding 은 고차원의 이산적(discrete) 데이터를 저차원의 연속적(continuous) 벡터 공간으로 변환하는 기술
- 컴퓨터가 텍스트, 이미지, 음성 등의 복잡한 데이터를 수치적으로 처리 가능

## ❖ 핵심 아이디어

- 의미론적 유사성 : 비슷한 의미를 가진 데이터는 벡터 공간에서 가까운 위치에 배치
- 차원 축소 : 고차원 데이터를 효율적으로 표현
- 연산 가능성 : 벡터 연산을 통한 의미 관계 표현



## ❖ One-Hot Encoding

- 단어사전: ["사과", "바나나", "오렌지", "자동차"]
  - "사과"  $\rightarrow [1, 0, 0, 0]$
  - "바나나"  $\rightarrow [0, 1, 0, 0]$
  - "오렌지"  $\rightarrow [0, 0, 1, 0]$
  - "자동차"  $\rightarrow [0, 0, 0, 1]$
- 문제점:
  - 차원의 저주(Curse of Dimensionality)
  - 의미적 유사성 반영 불가
  - 희소성(Sparsity) 문제

# 전통적인 텍스트 표현 방법

## ❖ Bag of Words (BoW)

- 문서를 단어의 빈도로 표현하는 방법

문서1: "사과는 빨간색이다"

문서2: "바나나는 노란색이다"

- 단어사전: ["사과", "바나나", "빨간색", "노란색", "이다"]

문서1 → [1, 0, 1, 0, 1]

문서2 → [0, 1, 0, 1, 1]

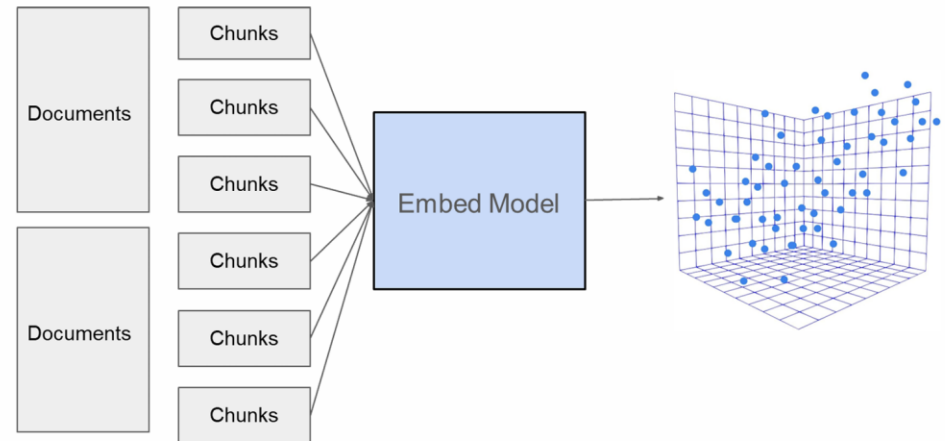
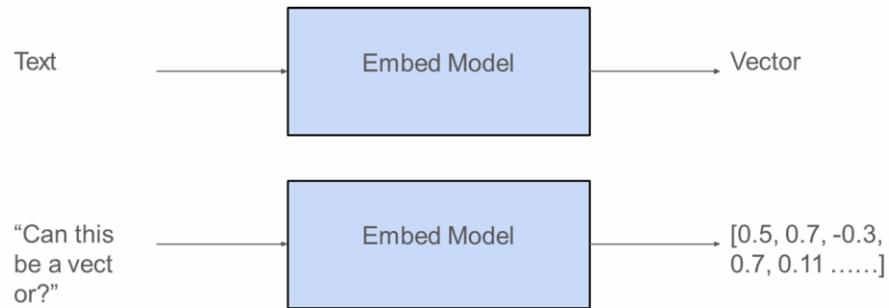
## ❖ TF-IDF (Term Frequency-Inverse Document Frequency)

- $TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$

$TF(t,d) = (\text{단어 } t \text{가 문서 } d \text{에 나타난 횟수}) / (\text{문서 } d \text{의 총단어수})$

$IDF(t) = \log(\text{전체 문서 수} / \text{단어 } t \text{가 포함된 문서 수})$

# 문장 및 문서 임베딩 : 문맥적 임베딩의 필요성





# 벡터 데이터 베이스란?

## ❖ 정의

- 고차원 벡터 데이터를 효율적으로 저장, 인덱싱, 검색할 수 있도록 설계된 전문 데이터베이스

## ❖ 등장 배경

- 전통적 검색의 한계
  - 키워드 기반 검색 : 정확한 단어 매칭만 가능
  - 의미적 유사성 반영 불가
  - 다중 모델 데이터 처리 한계

## ❖ 예시 :

- Query : “빨간 과일“
- 전통적 DB : “빨간“, “과일“ 키워드가 포함된 문서만 검색
- 벡터 DB : “사과“, “딸기“, “체리“ 등 의미적으로 유사한 항목 검색

# 전통적 데이터베이스 vs. 벡터 데이터베이스

| 측면     | 전통적 DB                | 벡터 DB                    |
|--------|-----------------------|--------------------------|
| 데이터 타입 | 정형 데이터 (숫자, 문자열, 테이블) | 고차원 벡터                   |
| 검색 방식  | 정확 매칭, 범위 검색          | 유사도 검색 (ANN)             |
| 인덱싱    | B-Tree, Hash          | LSH, HNSW, IVF           |
| 쿼리     | SQL, NoSQL            | 벡터 유사도                   |
| 확장성    | 수직적 확장                | 수평적 확장                   |
| 분야     | 전통적 비즈니스 applucation  | AI 검색, 추천시스템, 음성/ 이미지 검색 |

## 사용 사례 비교

### # 전통적 DB 쿼리

```
SELECT * FROM products
WHERE category = 'electronics' AND price BETWEEN 100 AND 500;
```

### # 벡터 DB 쿼리 (개념적)

```
SELECT * FROM products
WHERE vector_similarity(product_embedding, query_embedding) > 0.8
ORDER BY similarity DESC
LIMIT 10;
```

## 벡터 유사도 측정

- 유클리드 거리 (Euclidean Distance) : 최단 직선 거리
- 맨해튼 거리 (Manhattan Distance) : 모든 요소의 차이의 절대값의 합
- 코사인 유사도 (Cosine Similarity) : 벡터의 사잇각의 코사인 값
- 자카드 유사도 (Jaccard Similarity) : 아이템들을 0과 1의 바이너리 데이터로 만든 후, 두 벡터의 교집합과 합집합을 구하여 아이템 간의 유사성을 판단

[REF] WIKI > VectorDB > Implementations ([https://en.wikipedia.org/wiki/Vector\\_database](https://en.wikipedia.org/wiki/Vector_database))

| 구분     | Pinecone                             | FAISS                                   | Chroma                            |
|--------|--------------------------------------|-----------------------------------------|-----------------------------------|
| 출시 년도  | 2021                                 | 2017                                    | 2023                              |
| 개발사    | Pinecone Systems, Inc.               | Meta                                    | ChromaDB                          |
| 주요 목적  | <b>클라우드 기반</b> 벡터 데이터 검색 및 관리 (SaaS) | 벡터 데이터의 고속 검색 및 관리 (OpenSource)         | <b>벡터와 메타데이터를 관리</b> (OpenSource) |
| 성능     | 대규모 클러스터로 확장 가능, 실시간 벡터 검색 지원        | <b>GPU 기반</b> 의 고성능 벡터 검색               | 중소규모 데이터셋에서 빠르고 <b>간단</b> 하게 사용   |
| 확장성    | 데이터와 트래픽 증가 따라 자동 확장 지원              | 로컬 환경에서 실행되며, 확장 시 추가적인 설정 필요           | 로컬 데이터 관리 중심으로 확장성은 제한적           |
| 특징적 기능 | 클라우드 네이티브 <b>확장성</b>                 | Product Quantization(PQ)로 메모리 효율적 검색    | Python API로 쉽고 빠르게 사용 가능          |
| 장점     | 클라우드 기반으로 운영 부담 감소, 실시간 검색에 최적화      | GPU 기반의 <b>고성능 검색</b> , 다양한 인덱싱 알고리즘 제공 | 로컬 데이터 처리 및 관리가 간단                |
| 단점     | SaaS 모델로 관련 비용 발생                    | 배포 및 설정 복잡 (특히 대규모 데이터에 대한 관리)          | 대규모 데이터에서의 확장성과 실시간 처리에 한계        |

# RAG(Retrieval-Augmented Generation)

자연어 처리 분야에서 사용되는 기술로, 정보 검색과 생성을 결합한 인공지능 모델  
특히 복잡하고 정보가 필요한 질문에 답변하기 위해 설계  
크게 두 단계로 구성: 정보 검색(retrieval) 단계와 텍스트 생성(generation) 단계  
이러한 결합은 모델이 질문에 대해 보다 정확하고 관련성 높은 답변을 생성할 수 있도록 함

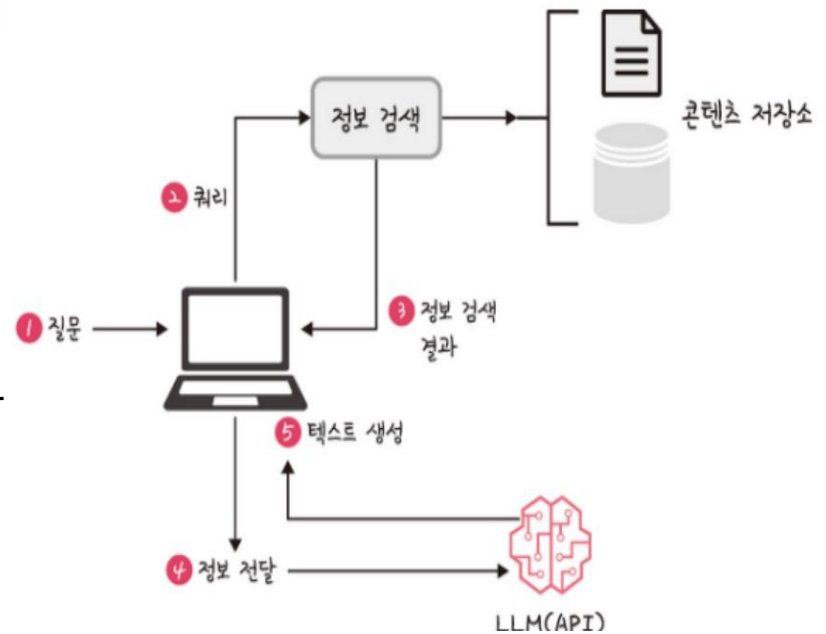
# RAG 단계

## ❖ 정보 검색 단계:

1. 질문: 사용자로부터 질문이 입력
2. 쿼리(문서 검색): 모델은 대규모의 문서 데이터베이스나 콘텐츠 저장소에서 질문과 관련된 문서나 정보를 검색
3. 정보 검색 결과: 검색 결과 중에서 가장 관련성 높은 문서와 사용자의 질문을 결합하여 LLM에 전달

## ❖ 텍스트 생성 단계:

4. 정보 전달: 선택된 문서의 내용이 모델에 전달(이 단계에서 모델은 문서의 정보를 활용하여 질문에 대한 의미를 이해)
5. 텍스트 생성: 모델은 이제 전달받은 정보를 바탕으로 질문에 대한 답변을 생성(이 과정은 LLM에 의해 처리되며, 문서에서 얻은 지식과 모델이 이미 학습한 정보를 결합하여 답변을 만들고 이후 생성된 답변을 사용자에게 제공)



## ❖ 랭체인이 도움이 되는 경우:

랭체인은 LLM(Large Language Model)에 외부의 '지식'이나 '계산 능력'을 활용하고 싶을 때 유용합니다.

LLM이 자신이 학습한 것만으로 대화하는 대신, '책'이나 '프로그램'을 통해 외부의 '지식'이나 '계산 능력'을 활용할 수 있게 하는 것이 랭체인의 역할입니다.

## ■ 예시:

예를 들어, 랭체인을 통해 LLM에 웹 검색 기능을 연결하면, LLM이 자신의 지식만으로는 충분한 답변을 할 수 없는 경우 웹 검색을 통해 최신 정보를 얻어 답변할 수 있게 됩니다.

랭체인 공식 문서 [https://python.langchain.com/docs/get\\_started/introduction.html](https://python.langchain.com/docs/get_started/introduction.html)

랭체인은 LLM을 활용한 애플리케이션 개발에 도움이 되는 다양한 모듈을 제공

모듈은 개별적으로 사용할 수 있을 뿐더러 여러 개를 조합해서 복잡한 애플리케이션을 구축할 수도 있음

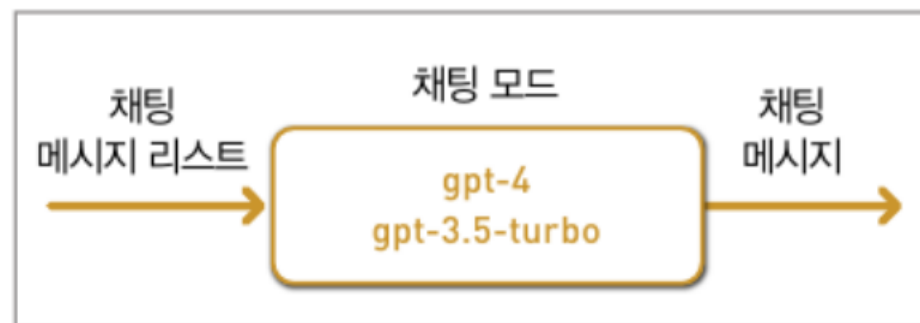
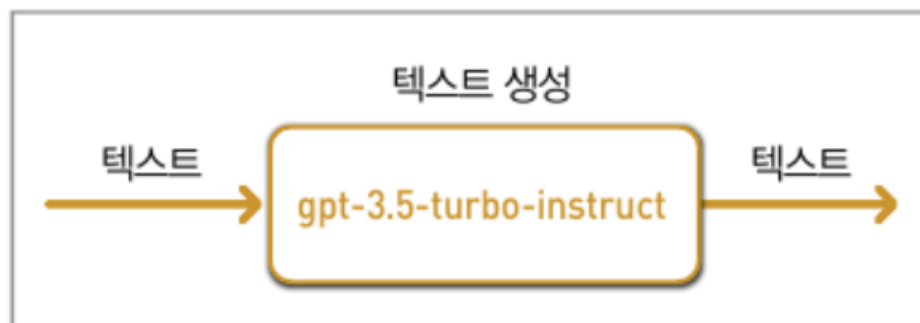
- LLM: LLM 호출을 위한 공통 인터페이스
- 프롬프트 템플릿: 사용자 입력에 따른 프롬프트 생성
- 체인: 여러 LLM과 프롬프트의 입력출력을 연결
- 에이전트: 사용자의 요청에 따라 어떤 기능을 어떤 순서로 실행할 것인지 결정
- 도구: 에이전트가 수행하는 특정 기능메모리: 체인 및 에이전트의 메모리 보유



# 랭체인 모듈

- LLM
- 프롬프트 템플릿
- 체인
- 에이전트
- 도구
- 메모리

# LLM:LLM 호출을 위한 공통 인터페이스



# 프롬프트 템플릿



- PromptTemplate: 프롬프트 템플릿 기본 클래스
- FewShotPromptTemplate: 답변 예시가 있는 프롬프트 템플릿

```
# 여러 개의 입력 변수가 있는 프롬프트 템플릿 만들기
multiple_input_prompt = PromptTemplate(
    input_variables=["adjective", "content"],
    template="{adjective} {content}이라고 하면?"
)
```

| 매개변수            | 설명           |
|-----------------|--------------|
| input_variables | 입력 변수 이름의 배열 |
| template        | 템플릿 문자열      |

# 프롬프트 템플릿(FewShot)

```
from langchain.prompts import FewShotPromptTemplate
```

```
# 답변 예시 준비
```

```
examples = [  
    {"input": "밝은", "output": "어두운"},  
    {"input": "재미있는", "output": "지루한"},  
]
```

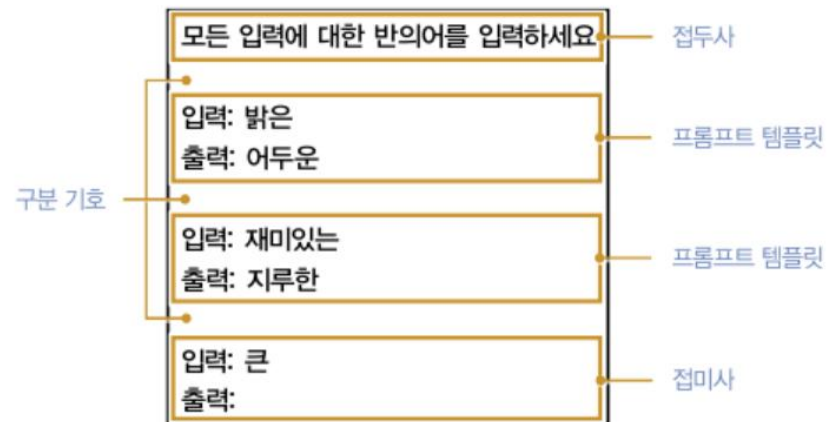
```
# 프롬프트 템플릿 만들기
```

```
example_prompt = PromptTemplate(  
    input_variables=["input","output"],  
    template="입력: {input}\n출력: {output}",  
)
```

```
# 답변 예시를 포함한 프롬프트 템플릿 만들기
```

```
prompt_from_string_examples = FewShotPromptTemplate(  
    examples=examples, # 답변 예시  
    example_prompt=example_prompt, # 프롬프트 템플릿  
    prefix=" 모든 입력에 대한 반의어를 입력하세요", # 접두사  
    suffix=" 입력: {adjective}\n출력:", # 접미사  
    input_variables=["adjective"], # 입력 변수  
    example_separator="\n\n" # 구분 기호  
)
```

| 매개변수              | 설명       |
|-------------------|----------|
| examples          | 답변 예시    |
| example_prompt    | 프롬프트 템플릿 |
| prefix            | 접두사      |
| suffix            | 접미사      |
| input_variables   | 입력 변수    |
| example_separator | 구분 기호    |



## ● 제네릭 체인

- LLMChain: 사용자 입력을 기반으로 프롬프트 템플릿으로 프롬프트를 생성해서 LLM을 호출합니다.
- SimpleSequentialChain: 입출력이 하나씩 있는 여러 개의 체인을 연결합니다.
- SequentialChain: 여러 개의 입출력을 가진 체인을 연결합니다.

## ● 인덱스 체인

- RetrievalQA: 질의응답을 수행합니다.
- RetrievalQAWithSourcesChain: 소스가 있는 질의응답을 수행합니다.
- SummarizeChain: 요약

- ConversationBufferMemory: 모든 대화 기록을 사용하는 메모리
- ConversationBufferWindowMemory: 최근 K번의 대화 기록을 사용하는 메모리
- ConversationTokenBufferMemory: 최신 K개 토큰의 대화 기록을 사용하는 메모리
- ConversationSummaryMemory: 대화 기록의 요약물을 사용하는 메모리
- ConversationSummaryBufferMemory: 최신 K개 토큰의 대화 내역 요약물을 사용하는 메모리
- ConversationEntityMemory: 대화 내 엔티티 정보를 저장하고 필요에 따라 사용하는 메모리
- ConversationKGMemory: 지식 그래프의 정보를 사용하는 메모리
- VectorStoreRetrieverMemory: 대화 기록을 벡터 데이터베이스에 저장해서 상위 K개의 유사한 정보를 사용하는 메모리

# 랭그래프 시스템 이해

- 랭그래프(LangGraph) 소개와 워크플로 구성

# LangGraph란?

상태를 가진 멀티 액터 애플리케이션을 구축하는 강력한 라이브러리

LangChain 위에 구축되어 LLM 기반 AI Agent를 위한 순환 그래프를 생성

상태를 가진, 순환적이고 멀티 액터 대형 언어 모델 애플리케이션을 위한 다재다능한 Python 라이브러리

## LangGraph의 핵심 장점

- **순환 그래프 토폴로지**: 선형 모델보다 유연하고 세밀한 Agent 동작 가능
- **높은 제어성**: 개발자가 고도로 제어 가능한 Agent 생성
- **상태 관리**: 대화 히스토리와 Agent 상태 추적
- **멀티 액터 워크플로우**: 여러 Agent가 협력하는 복잡한 워크플로우 구현



# LangGraph 핵심 구성 요소

## 1. State (상태)

- **정의:** 애플리케이션의 현재 스냅샷을 나타내는 공유 데이터 구조
- **역할:** 노드 간 �지를 따라 전달되어 한 노드의 출력을 다음 노드의 입력으로 전달
- **특징:** 전체 워크플로우의 백본 역할 수행

## 2. Nodes (노드)

- **정의:** 함수 또는 LangChain 구성 요소를 나타내는 핵심 요소
- **역할:** 각 노드는 LLM Agent를 나타내며, 특정 작업을 수행
- **기능:** 상태를 받아 처리하고 결과를 반환

## 3. Edges (엣지)

- **정의:** Agent 간 통신 채널
- **유형:**
  - **Simple Edges:** 단순 연결 (node1 → node2)
  - **Conditional Edges:** 조건부 분기 처리
  - **Entry/Exit Edges:** 시작점과 종료점 정의

**요청 분석:** 사용자 입력을 분석하여 필요한 도구 식별

**도구 호출:** 식별된 도구를 순차적 또는 병렬로 실행

**결과 통합:** 여러 도구의 결과를 통합하여 최종 응답 생성

**상태 업데이트:** 실행 결과를 상태에 반영

## Checkpointing

- 실행 중간 상태 저장
- 장애 복구 및 재시작 지원
- 대화 히스토리 유지

## Conditional Routing

- 상태에 따른 동적 경로 결정
- 복잡한 비즈니스 로직 구현
- 예외 처리 및 에러 핸들링

## Multi-Agent Coordination

- 여러 Agent 간 협업 구현
- 작업 분산 및 결과 통합
- 상호 의존성 관리

---

LangGraph 설치 및 기본 환경 구성

---

간단한 Tool Calling Agent 구현

---

상태 관리 및 엣지 연결 실습

---

조건부 분기 처리 구현

## 상태 기반 RAG 워크플로우

검색과 생성 과정을 노드로 분리

상태를 통한 중간 결과 공유

조건부 분기를 통한 동적 검색 전략

## RAG 전용 상태 설계

```
class RAGState(TypedDict):
```

```
    query: str
```

```
    documents: list
```

```
    retrieved_chunks: list
```

```
    context: str
```

```
    answer: str
```

```
    confidence: float
```

```
    metadata: dict
```

# RAG 워크플로우 노드 구성

## 1. Query Processing Node

- 사용자 쿼리 분석 및 전처리
- 키워드 추출 및 쿼리 확장
- 검색 전략 결정

## 2. Document Retrieval Node

- 벡터 데이터베이스 검색
- 하이브리드 검색 (벡터 + 키워드)
- 검색 결과 스코어링

## 3. Context Assembly Node

- 검색된 문서 순위 결정
- 컨텍스트 윈도우 최적화
- 중복 제거 및 정제

## 4. Answer Generation Node

- LLM 기반 답변 생성
- 컨텍스트 기반 응답 구성
- 신뢰도 점수 계산

# Agent RAG 필요성 및 주요 특징 이해

## •Agent RAG의 필요성

- 정보의 실시간성과 정확성 요구 증가
- 대용량 데이터 처리 및 활용 필요
- 개인화된 서비스 제공 요구

## •주요 특징

- 실시간 정보 검색 및 활용
- 컨텍스트 유지 및 연속성 보장
- 다양한 데이터 형식 지원
- 확장성 및 유연성

## •기존 시스템과의 차이점

- 정적 vs 동적 지식 활용
- 단일 vs 다중 데이터 소스 연동
- 배치 vs 실시간 처리

# 다양한 모델을 활용한 자동검색 및 답변하는 AI 비서 만들기

- 랭그래프(LangGraph) 활용 검색시스템 구축
- 검색 결과를 바탕으로 답변하는 AI agent 구축